

**T.C.
SAKARYA ÜNİVERSİTESİ
FEN BİLİMLERİ ENSTİTÜSÜ**

**URL ANALİZİ VE MAKİNE ÖĞRENMESİ TABANLI KİMLİK AVI
TESPİTİ VE TARAYICI EKLENTİSİ GELİŞTİRİLMESİ**

YÜKSEK LİSANS TEZİ

Mustafa Melih TÜFEKCİOĞLU

Bilgisayar Mühendisliği Anabilim Dalı

Siber Güvenlik Bilim Dalı

TEMMUZ 2026

**T.C.
SAKARYA ÜNİVERSİTESİ
FEN BİLİMLERİ ENSTİTÜSÜ**

**URL ANALİZİ VE MAKİNE ÖĞRENMESİ TABANLI KİMLİK AVI
TESPİTİ VE TARAYICI EKLENTİSİ GELİŞTİRİLMESİ**

YÜKSEK LİSANS TEZİ

Mustafa Melih TÜFEKCİOĞLU

Bilgisayar Mühendisliği Anabilim Dalı

Siber Güvenlik Bilim Dalı

Tez Danışmanı: Dr. Öğr. Üyesi Musa BALTA

TEMMUZ 2026

Mustafa Melih TÜFEKCİOĞLU tarafından hazırlanan “ URL ANALİZİ VE MAKİNE ÖĞRENMESİ TABANLI KİMLİK AVI TESPİTİ VE TARAYICI EKLENTİSİ GELİŞTİRİLMESİ ” adlı tez çalışması 29.07.2026 tarihinde aşağıdaki jüri tarafından oy birliği/oy çokluğu ile Sakarya Üniversitesi Fen Bilimleri Enstitüsü Bilgisayar Mühendisliği Anabilim Dalı Siber Güvenlik Bilim Dalı’nda Yüksek Lisans tezi olarak kabul edilmiştir.

Tez Jürisi

Jüri Başkanı :	Doç. Dr. Murat İSKEFİYELİ
	Sakarya Üniversitesi
Jüri Üyesi :	Dr. Öğr. Üyesi Musa BALTA (Danışman)
	Sakarya Üniversitesi
Jüri Üyesi :	Doç. Dr. Selman Hızal
	Sakarya Uygulamalı Bilimler Üniversitesi

ETİK İLKE VE KURALLARA UYGUNLUK BEYANNAMESİ

Sakarya Üniversitesi Fen Bilimleri Enstitüsü Lisansüstü Eğitim-Öğretim Yönetmeliğine ve Yükseköğretim Kurumları Bilimsel Araştırma ve Yayın Etiği Yönergesine uygun olarak hazırlamış olduğum” URL ANALİZİ VE MAKİNE ÖĞRENMESİ TABANLI KİMLİK AVI TESPİTİ VE TARAYICI EKLENTİSİ GELİŞTİRİLMESİ” başlıklı tezin bana ait, özgün bir çalışma olduğunu; çalışmamın tüm aşamalarında yukarıda belirtilen yönetmelik ve yönergeye uygun davrandığımı, tezin içerdiği yenilik ve sonuçları başka bir yerden almadığımı, tezde kullandığım eserleri usulüne göre kaynak olarak gösterdiğimi, bu tezi başka bir bilim kuruluna akademik amaç ve unvan almak amacıyla vermediğimi ve 20.04.2016 tarihli Resmi Gazete’de yayımlanan Lisansüstü Eğitim ve Öğretim Yönetmeliğinin 9/2 ve 22/2 maddeleri gereğince Sakarya Üniversitesi’nin abonesi olduğu intihal yazılım programı kullanılarak Enstitü tarafından belirlenmiş ölçütlere uygun rapor alındığını, etik kurul onay belgesi aldığımı, çalışmamla ilgili yaptığım bu beyana aykırı bir durumun ortaya çıkması halinde doğabilecek her türlü hukuki sorumluluğu kabul ettiğimi beyan ederim.

(...../...../2026).

(imza)

Mustafa Melih TÜFEKCİOĞLU

TEŞEKKÜR

Yüksek lisans eğitimim ve bu tezin hazırlanması sürecinde bilgi ve deneyimleriyle bana rehberlik eden, akademik gelişimime önemli katkılar sağlayan ve her aşamada desteklerini esirgemeyen değerli danışmanım Dr. Öğr. Üyesi Musa Balta'ya en içten teşekkürlerimi sunarım.

Tez çalışmam boyunca maddi ve manevi desteklerini her zaman yanımda hissettiğim, sabırları ve fedakârlıklarıyla bugünlere gelmemde büyük pay sahibi olan anneme ve babama sonsuz teşekkür ederim.

Ayrıca, her zaman yanımda olan, motivasyonumu artıran ve desteklerini esirgemeyen kıymetli abilerime teşekkürlerimi sunarım.

Bu süreçte destek ve anlayışlarıyla bana güç veren geniş aileme de ayrıca teşekkür ederim.

Mustafa Melih TÜFEKCİOĞLU

İÇİNDEKİLER

Sayfa

ETİK İLKE VE KURALLARA UYGUNLUK BEYANNAMESİ	v
TEŞEKKÜR	vii
İÇİNDEKİLER	ix
KISALTMALAR	xi
TABLO LİSTESİ	xiii
ŞEKİL LİSTESİ	xv
ÖZET	xvii
SUMMARY	xix
1. GİRİŞ	1
1.1. Problem Tanımı	3
1.2. Tezin Amacı ve Kapsamı	4
1.3. Tez Organizasyonu	5
2. TEMEL BİLGİLER VE LİTERATÜR ARAŞTIRMASI	7
2.1. Web ve URL Güvenliği	7
2.2. Web Güvenliğinde Makine Öğrenmesi Algoritmaları	9
2.3. Lexical Analiz	13
2.4. Literatür Çalışmaları	14
3. URL ANALİZİ ve ÖZELLİK MÜHENDİSLİĞİ	21
3.1. URL Analizi Süreci ve Genel Yapı	21
3.2. Veri Kaynakları ve Toplama Yöntemi	22
3.3. Veri Ön İşleme Aşamaları	23
3.4. Lexical Özelliklerin Belirlenmesi	23
3.5. Özellik Seçimi ve Boyut Azaltma	24
4. MODEL GELİŞTİRME VE GERÇEK ZAMANLI SİSTEM TASARIMI ...	27
4.1. Kullanılan Makine Öğrenmesi Algoritmaları	27
4.2. Model Eğitimi, Doğrulama ve Optimizasyon	30
4.3. Gerçek Zamanlı Sistem Mimarisi	31
4.4. Tarayıcı Eklentisi Geliştirme Süreci	33
4.5. Güvenlik ve Performans Değerlendirme	35
5. SONUÇLAR VE DEĞERLENDİRME	39
5.1. Deneysel Bulgular	39
5.2. Model Performans ve Karşılaştırmalar	43
5.3. Bulguların Yorumu	45
5.4. Tarayıcı Eklentisi Saha Testi ve Gerçek Zamanlı Performans Analizi	48
KAYNAKLAR	51
ÖZGEÇMİŞ	57

KISALTMALAR

API	: Application Programming Interface (Uygulama Programlama Arayüzü)
CMS	: Content Management System (İçerik Yönetim Sistemi)
CNN	: Convolutional Neural Networks (Evrişimli Sinir Ağları)
CPU	: Central Processing Unit (Merkezi İşlem Birimi)
CV	: Cross-Validation (Çapraz Doğrulama)
DLP	: Data Loss Prevention (Veri Kaybı Önleme)
DNS	: Domain Name System (Alan Adı Sistemi)
GBM	: Gradient Boosting Machines (Gradyan Artırma Makineleri)
GPT	: Generative Pre-trained Transformer (Üretken Önceden Eğitilmiş Dönüştürücü)
HTTP/HTTPS	: Hypertext Transfer Protocol/Secure (Hiper Metin Transfer Protokolü/Güvenli)
IDS/IPS	: Intrusion Detection/Prevention System (Saldırı Tespit/Önleme Sistemi)
IP	: Internet Protocol (İnternet Protokolü)
JSON	: JavaScript Object Notation (JavaScript Nesne Gösterimi)
KNN	: K-Nearest Neighbors (K-En Yakın Komşu)
LSTM	: Long Short-Term Memory (Uzun Kısa Süreli Bellek)
MB	: Megabyte (Megabayt)
MLP	: Multi-layer Perceptron (Çok Katmanlı Algılayıcı)
NLP	: Natural Language Processing (Doğal Dil İşleme)
ROC-AUC	: Receiver Operating Characteristic - Area Under Curve (Alıcı İşletim Karakteristiği - Eğri Altındaki Alan)
SQL	: Structured Query Language (Yapılandırılmış Sorgu Dili)
SSL/TLS	: Secure Sockets Layer (Transport Layer Security (Güvenli Yuva Katmanı / Taşıma Katmanı Güvenliği)
SVM	: Support Vector Machines (Destek Vektör Makineleri)
TLD	: Top-Level Domain (Üst Düzey Alan Adı)
URL	: Uniform Resource Locator (Tekdüzen Kaynak Bulucu)
WAF	: Web Application Firewall (Web Uygulama Güvenlik Duvarı)

WHOIS	: Alan adı kayıt bilgilerini sorgulama protokolü
XGBoost	: Extreme Gradient Boosting (Aşırı Gradyan Artırma)
XSS	: Cross-Site Scripting (Siteler Arası Komut Dosyası Çalıştırma)

TABLO LİSTESİ

Sayfa

Tablo 2.1. Literatür çalışmaları	17
Tablo 3.1. Özellik kategorileri ve tanımı.....	26
Tablo 5.1. Model eğitim sonuçları	41
Tablo 5.2. Özellik azaltımı olmadan model eğitim sonuçları	42
Tablo 5.3. Tarayıcı eklentisi saha testi başarı ve süre metrikleri	48

ŞEKİL LİSTESİ

Sayfa

Şekil 1.1. Kimlik avı yaşam döngüsü	3
Şekil 2.1. URL bileşenleri.....	7
Şekil 2.2. Destek vektör makinelerinde optimal ayrım düzlemi ve destek vektörleri	11
Şekil 2.3. Yapay sinir ağları.....	12
Şekil 2.4. Token özellikleri.....	14
Şekil 3.1. Korelasyon sonuçları	25
Şekil 4.1. Algoritma kullanım oranı	27
Şekil 4.2. Banner.....	35

URL ANALİZİ VE MAKİNE ÖĞRENMESİ TABANLI KİMLİK AVI TESPİTİ VE TARAYICI EKLENTİSİ GELİŞTİRİLMESİ

ÖZET

Günümüzde internet ve dijital teknolojiler, hayatın hemen hemen her alanına derinlemesine entegre olmuş durumdadır. Kullanıcılar; bankacılık işlemlerinden alışverişe, eğitimden sosyal etkileşime, kamu hizmetlerinden kurumsal iletişime kadar birçok faaliyetini artık web tabanlı platformlar üzerinden gerçekleştirmektedir. Bu denli yaygın ve yoğun kullanım, siber saldırganlar için oldukça geniş bir hedef yüzeyi oluşturmakta ve özellikle kimlik avı (phishing) saldırıları, en sık karşılaşılan ve en yıkıcı sonuçlar doğurabilen tehdit türlerinden biri hâline gelmektedir. Oltalama saldırıları, kullanıcıları görsel ve içerik açısından meşru sitelere son derece benzeyen sahte web sitelerine yönlendirerek şifre, kredi kartı bilgisi gibi hassas verilerin ele geçirilmesini amaçlamakta; bu durum hem bireyler hem de kurumlar için ciddi finansal kayıplara ve güvenlik risklerine yol açmaktadır.

Geleneksel güvenlik yaklaşımları çoğunlukla kara listeleme, imza tabanlı tespit ve manuel analiz yöntemlerine dayanmaktadır. Ancak bu yöntemler, sürekli değişen, kendini güncelleyen ve yeni varyasyonlarla ortaya çıkan oltalama saldırılarına karşı yetersiz kalabilmektedir. Özellikle daha önce hiç görülmemiş, yeni üretilen (zero-day) zararlı URL'lerin tespiti, klasik yöntemlerle oldukça zorlaşmakta ve tepki süresi saldırganların lehine işlemektedir. Bu nedenle, değişen tehdit ortamına uyum sağlayabilen, öğrenebilir ve genellenebilir sistemlere olan ihtiyaç giderek artmaktadır.

Bu çalışmada, phishing saldırılarının tespiti amacıyla URL tabanlı bir analiz yaklaşımı ele alınmış ve makine öğrenmesi yöntemleri kullanılarak kapsamlı bir sınıflandırma sistemi geliştirilmiştir. Çalışma kapsamında güncel ve kapsamlı bir veri kaynağı olan StealthPhisher 2025 veri seti kullanılmış; eksik ve gürültülü verilerin temizlenmesi de dâhil olmak üzere veri ön işleme adımları uygulanmış ve özellikle lexical (metinsel) özellikler çıkarılarak veri seti model eğitime uygun hâle getirilmiştir. Ayrıca özellik seçimi sürecinde korelasyon analizi ve Mutual Information yöntemleri kullanılarak veri boyutu azaltılmış, gereksiz ve birbirini tekrar eden öznelilikler elenmiş, böylece model performansı optimize edilmiştir.

Geliştirilen sistemde farklı makine öğrenmesi algoritmaları kullanılarak modeller eğitilmiş, çapraz doğrulama süreçleri gerçekleştirilmiş ve elde edilen sonuçlar çeşitli performans metrikleri açısından karşılaştırılmıştır. Elde edilen bulgular, önerilen özellik mühendisliği yaklaşımının model başarımını anlamlı şekilde artırdığını göstermektedir.

Bununla birlikte çalışma yalnızca teorik bir model geliştirmeye sınırlı kalmamış, aynı zamanda gerçek zamanlı kullanım senaryosu da ele alınmıştır. Bu kapsamda geliştirilen modelin bir tarayıcı eklentisi aracılığıyla entegre edilmesi sağlanmış ve kullanıcıların ziyaret ettiği URL'lerin anlık olarak analiz edilerek zararlı olup olmadığının belirlenmesi hedeflenmiştir. Bu sayede sistemin pratikte uygulanabilirliği

somut biçimde gösterilmiş, kullanıcı odaklı ve düşük gecikmeli bir siber güvenlik çözümü sunulmuştur.

Sonuç olarak bu çalışma, phishing saldırılarının tespitinde makine öğrenmesi tabanlı yaklaşımların etkinliğini ortaya koymakta; önerilen özellik mühendisliği yöntemleri ve gerçek zamanlı sistem entegrasyonu ile hem literatüre hem de uygulamaya yönelik somut bir katkı sağlamaktadır.

Anahtar Kelimeler: Siber güvenlik, özellik mühendisliği, makine öğrenmesi, zararlı URL tespiti, URL tabanlı analiz, web güvenliği

DEVELOPMENT OF A BROWSER EXTENSION FOR PHISHING DETECTION USING URL ANALYSIS AND MACHINE LEARNING

SUMMARY

Today, the internet and digital technologies have become deeply integrated into nearly every aspect of daily life. Users conduct banking transactions, online shopping, educational activities, social interactions, public services, and corporate communications through web-based platforms. This widespread dependence has also expanded the attack surface available to cyber attackers. Among the threats that exploit this environment, phishing attacks are one of the most common and damaging forms of cybercrime. Phishing attacks attempt to deceive users by directing them to malicious websites or links that imitate legitimate services. Such websites may reproduce the appearance, naming conventions, and domain structures of trusted platforms, making the deception difficult to recognize. Consequently, attackers can obtain sensitive information such as usernames, passwords, financial information, and credit card details, creating serious security and financial risks for both individuals and organizations.

Traditional security mechanisms used to detect phishing websites commonly depend on blacklists, signatures, predefined rules, and manual analysis. Although these mechanisms can be effective against previously identified threats, their effectiveness decreases when attackers create new URLs, modify existing URL structures, or use techniques intended to evade known patterns. In particular, previously unseen or newly generated zero-day malicious URLs may not yet exist in a blacklist and therefore cannot be reliably identified through a simple lookup operation. The dynamic nature of phishing attacks creates a need for detection mechanisms that can identify suspicious patterns rather than relying exclusively on previously observed examples. A practical system must also provide its decision with sufficiently low latency so that users can be warned before they interact further with a malicious resource.

In this study, a URL-based analysis approach was adopted for phishing detection, and a machine learning based classification system was developed. The main objective is to determine whether a URL is legitimate or phishing by analyzing information contained directly in the URL itself. The proposed approach does not depend on external information sources such as WHOIS, DNS queries, or web-page content during the real-time detection stage. This design reduces the amount of information that must be collected and processed and supports a lightweight analysis pipeline suitable for browser-based use. The study therefore combines machine learning based classification with a real-time browser extension in order to address both detection accuracy and response-time requirements.

The StealthPhisher 2025 dataset was used as the primary data source. The dataset contains 336,750 URL records and has a balanced distribution of phishing and legitimate samples. Before model training, preprocessing operations were performed to obtain a consistent and suitable data structure. Attributes that were unnecessary for the scope of the study or depended on external information were excluded. Missing,

erroneous, or inconsistent records were checked and cleaned, data types were standardized, and undefined values that could occur in ratio-based calculations were handled. These operations ensured that the resulting data could be processed consistently by the selected machine learning algorithms and that the analysis remained focused on information obtainable directly from URLs.

A major component of the proposed system is lexical feature engineering. In total, 54 features were automatically extracted from URL strings using Python. These features represent structural, statistical, and character-level properties of URLs, including URL and component lengths, domain and subdomain characteristics, special-character frequencies, digit and letter distributions, character repetition patterns, entropy and complexity measures, and other URL-based descriptors. Representing each URL as a numerical feature vector transforms the raw character sequence into a form that conventional machine learning algorithms can process efficiently. Because the required information is available directly from the URL, this approach also supports rapid analysis without downloading or examining the complete web page.

The initial feature set was intentionally broad so that potentially informative URL characteristics would not be excluded at the beginning of the study. However, a large feature set may contain redundant information, increase computational cost, and reduce the efficiency of learning. Therefore, a two-stage feature selection strategy was applied. First, correlation analysis was used to identify highly related features. Redundant attributes were removed so that the models would not repeatedly learn similar information. Second, Mutual Information was used to measure the information contribution of each feature with respect to the target class, namely phishing or legitimate. This method can capture informative relationships that are not necessarily linear. The combined procedure produced a more compact and discriminative feature subset while reducing unnecessary computational overhead.

After preprocessing and feature selection, eight machine learning algorithms were trained and compared: XGBoost, Random Forest, Logistic Regression, Decision Tree, Gradient Boosting, K-Nearest Neighbors (KNN), Linear Support Vector Machine (Linear SVM), and Naive Bayes. Model evaluation was not limited to accuracy. Test accuracy, precision, recall, F1-score, and ROC-AUC were used to assess classification performance. Cross-validation accuracy, F1-score, and ROC-AUC were also calculated to evaluate model stability and generalization. In addition, training time, prediction time, memory usage, and CPU usage were considered because computational efficiency is an important requirement for a real-time browser security system.

The experimental results showed that the models achieved very high classification performance after feature selection. XGBoost provided the most balanced overall performance. With the optimized feature set, XGBoost achieved a test accuracy of 0.9976, precision of 0.9995, recall of 0.9958, F1-score of 0.9977, and ROC-AUC of 0.9988. Its cross-validation accuracy, F1-score, and ROC-AUC were 0.9979, 0.9980, and 0.9990, respectively. Its training time was approximately 0.86 seconds, prediction time was approximately 0.03 seconds, and memory usage was 103.65 MB. These results indicate that XGBoost provided a strong balance between predictive performance, stability, and computational cost.

The effect of feature selection was also examined by training the models without reducing the feature set. Classification accuracy remained high in both scenarios; however, several models required more computational resources when the feature set

was not reduced. For example, XGBoost maintained a test accuracy of 0.9976, while its training time increased from approximately 0.86 seconds to 0.90 seconds and its memory usage increased from 103.65 MB to 105.94 MB. Similar differences were observed for other algorithms. Therefore, the contribution of the feature selection process is not simply a large increase in accuracy; it provides a more compact representation that preserves high predictive performance while reducing unnecessary computational overhead and supporting more efficient deployment.

The selected model was subsequently integrated into a Chrome browser extension to evaluate the proposed approach in a real-time usage scenario. The extension automatically captures the URL visited by the user, performs the required lexical feature extraction, and submits the resulting feature vector to the trained classification model. The model then determines whether the URL is legitimate or malicious, and the extension provides an appropriate visual warning when a threat is detected. This structure creates an additional security layer that operates without requiring the user to manually copy or submit URLs for analysis. The integration therefore demonstrates the practical application of the machine learning model beyond an offline experimental environment.

A field test was conducted to evaluate the practical behavior of the browser extension. The evaluation used URL samples originating from the thesis dataset and external sources, with some URLs included in both groups to examine consistency. The final test contained 177 URL instances. The system produced 176 correct predictions and one incorrect prediction, corresponding to an accuracy of 99.44%. Precision was 1.000, recall was 0.987, and the F1-score was 0.9935. The test produced 76 true positives, 100 true negatives, zero false positives, and one false negative. The absence of false positives in the evaluated sample is important for user experience because legitimate websites were not incorrectly flagged as malicious.

Overall, the findings demonstrate that URL lexical analysis combined with machine learning can provide an effective approach to phishing detection without requiring direct analysis of web-page content. The use of 54 URL-derived features, followed by correlation-based reduction and Mutual Information based feature selection, produced a compact feature representation suitable for classification. Among the evaluated algorithms, XGBoost provided the most balanced combination of accuracy, stability, and computational efficiency. The close relationship between test and cross-validation results further supports the generalization capability of the developed models.

In conclusion, this study presents an end-to-end phishing URL detection approach that combines data preprocessing, lexical feature engineering, feature selection, machine learning classification, and real-time browser integration. The proposed system moves beyond conventional blacklist-based detection by learning patterns from URL structures and applying the learned model to URLs encountered during browsing. The experimental results demonstrate very high classification performance, while the browser-extension field test confirms the practical feasibility of low-latency URL analysis. The study therefore provides a concrete contribution to both academic research on machine learning based phishing detection and the development of a user-oriented cybersecurity application. Future improvements can focus on continuously updated training data, broader external validation, and further investigation of previously unseen phishing patterns.

Keywords: Cybersecurity, feature engineering, machine learning, malicious URL detection, URL-based analysis, web security.

1. GİRİŞ

Günümüzde internetin yaygınlaşması, kullanıcıların çevrimiçi işlemlerini kolaylaştırmaktadır. Ancak bu durum siber saldırıların da artmasına neden olmuştur. Yapılan araştırmalar, 2014–2023 yılları arasında küresel çapta bildirilen siber saldırı sayısının yaklaşık %300 oranında arttığını ve bu saldırıların dünya ekonomisine verdiği yıllık zararın 8–10 trilyon dolar seviyesine ulaştığını göstermektedir [1-2]. Bu saldırılar arasında en yaygın ve zararlı olanlardan biri oltalama (phishing), başka bir deyişle kimlik avı saldırılarıdır. Kimlik avı saldırıları, kullanıcıların güvenini kötüye kullanarak kişisel veya finansal bilgilerini elde etmeyi amaçlayan dolandırıcılık yöntemleridir. Bu saldırılar, genellikle sahte e-postalar, web siteleri veya bağlantılar aracılığıyla gerçekleştirilmektedir. Özellikle finansal kurumlar, sosyal medya platformları ve e-ticaret siteleri, saldırganların sıklıkla hedef aldığı alanlardır. Güncel güvenlik raporları, tüm siber saldırıların yaklaşık %36–40'ının phishing kaynaklı olduğunu ve veri ihlallerinin önemli bir kısmının bu saldırılar sonucu gerçekleştiğini ortaya koymaktadır [3]. Bu nedenle özellikle finansal kurumlar, sosyal medya platformları ve e-ticaret siteleri, saldırganların en sık hedef aldığı alanlar arasında yer almaktadır.

Kimlik avı saldırılarının tespit edilmesi, siber güvenlik alanında kritik bir öneme sahiptir. Geleneksel imza tabanlı veya kara liste tabanlı yaklaşımlar, yeni veya değiştirilmiş saldırı URL'lerini tespit etmede yetersiz kalmaktadır. Makine öğrenmesi tabanlı yaklaşımların temelini oluşturan lojistik regresyon gibi olasılıksal sınıflandırma yöntemleri [4,5] ve sınıflar arasındaki ayrım düzlemini maksimize etmeye dayanan destek vektör makineleri [6], bu tür problemlerin çözümünde yaygın olarak kullanılan teknikler arasında yer almaktadır. Bu yöntemlerin phishing tespiti problemine uygulanmasıyla birlikte, makine öğrenmesi ve derin öğrenme tabanlı yaklaşımlar son yıllarda önemli bir araştırma alanı haline gelmiştir [7-9]. Bu yöntemler, URL'lerin yapısal ve anlamsal özelliklerinden yararlanarak kötü niyetli bağlantıları tespit etmede yüksek doğruluk oranlarına ulaşabilmektedir.

Literatürdeki çalışmalara göre kimlik avı tespitinde kullanılan en temel yaklaşım, URL’lerin lexical (sözcüksel) özelliklerinin analiz edilmesidir. Lexical analiz, bir URL’nin içeriğinde yer alan karakter dizilimlerini, uzunlukları, sembolleri, alt alan adlarını ve belirli anahtar kelimeleri inceleyerek potansiyel tehditleri belirlemeyi amaçlar. Bu özelliklerden elde edilen veriler, makine öğrenmesi modelleri tarafından işlenerek “güvenli” veya “tehlikeli” olarak sınıflandırılır. Böylece, daha önce kara listede bulunmayan fakat yapısal olarak benzer özellikler taşıyan kimlik avı URL’leri tespit edilebilir [10-12].

Bununla birlikte, yapılan çalışmaların veya geliştirilen sistemlerin çoğu çevrimdışı (offline) analizler üzerine odaklanmıştır. Gerçek zamanlı sistemlerin azlığı, tespit edilen saldırıların kullanıcıya ulaşmadan önce engellenmesini zorlaştırmaktadır. Bu eksiklikten hareketle, bu tez çalışmasında gerçek zamanlı URL analizi gerçekleştiren, makine öğrenmesi tabanlı bir sınıflandırma modeli geliştirilmiş ve tarayıcı eklentisi aracılığıyla gerçek zamanlı uygulanabilir bir sistem tasarlanmıştır.

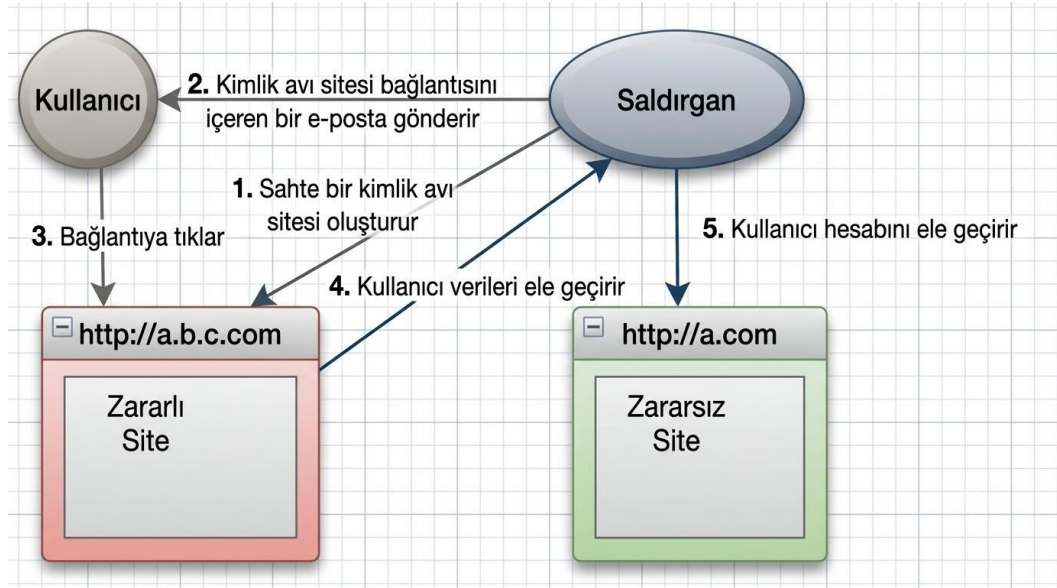
Geliştirilen sistem, kullanıcının ziyaret ettiği URL’yi tarayıcı üzerinden otomatik ve çalışma anında alır, lexical analiz işlemini gerçekleştirir ve eğitilmiş model aracılığıyla bağlantının güvenilirlik durumunu değerlendirir. Modelin çıktısına göre kullanıcı, ziyaret ettiği sayfanın güvenli olup olmadığı konusunda anında bilgilendirilir. Bu yapı, kullanıcı etkileşimi gerektirmeden çalışan, hızlı ve kullanıcı dostu bir güvenlik katmanı sunmaktadır.

Bu çalışmanın temel amacı, kimlik avı saldırılarını gerçek zamanlı olarak tespit edebilen, yüksek doğruluk oranına sahip, uygulanabilir, güvenli ve hızlı bir çözümü geliştirmektir. Çalışmanın çıktısı olarak oluşturulan tarayıcı eklentisi, kullanıcıların günlük internet kullanımında ek bir koruma katmanı sağlayarak siber güvenlik farkındalığını artırmayı hedeflemektedir.

Bu tez çalışması, siber güvenlik alanında lexical analiz ve yapay zeka tabanlı tespit yöntemlerinin bütünleştirilmesiyle oluşturulan gerçek zamanlı bir sistem önermektedir. Bu yönüyle hem akademik katkı hem de pratik uygulama açısından özgün bir çalışma niteliği taşımaktadır.

1.1. Problem Tanımı

Günümüzde internetin yaygınlaşmasıyla birlikte, kullanıcılar çevrimiçi platformlar üzerinden kişisel, finansal ve kurumsal işlemlerini sıklıkla gerçekleştirmektedir. Bu durum, dijital ortamda dolaşan hassas verilerin miktarını artırırken, kötü niyetli aktörlerin saldırı yüzeyini ve saldırı tekniklerini de genişletmiştir. Bu saldırı türlerinden en yaygın olanlardan biri kimlik avı (phishing) saldırılarıdır. Aşağıdaki şekilde kimlik avı saldırıları yaşam döngüsünün örneği verilmiştir.



Şekil 1.1. Kimlik avı yaşam döngüsü [13]

Kimlik avı saldırıları, kullanıcıları kandırarak kişisel bilgilerini (örneğin kullanıcı adı, parola, kredi kartı bilgileri) ele geçirmeyi amaçlayan sahte web siteleri aracılığıyla gerçekleştirilir. Genellikle e-posta veya sosyal medya üzerinden paylaşılan bağlantılar, kullanıcıları görünüşte güvenilir ancak gerçekte kötü niyetli web sitelerine yönlendirir.

Bu siteler çoğu zaman orijinal sitelerin tasarımını ve alan adını taklit ederek kullanıcıların fark etmesini zorlaştırır. Bu durum çoğu zaman kullanıcının doğru sitede olduğunu sanmasını sağlayarak kullanıcının bilgilerini çalmayı hedefler.

Geleneksel güvenlik çözümleri, kimlik avı sitelerini tespit etmek için kara listeler (black list) veya imza tabanlı yöntemlerden yararlanır. Ancak bu yöntemler, yalnızca daha önce tespit edilmiş saldırılara karşı etkili olup, yeni veya değişken URL yapısına sahip saldırılarda yetersiz kalmaktadır. Günümüzde saldırganlar, URL'lerde küçük karakter değişiklikleri, alt alan adı manipülasyonları veya link kısaltma servisleri gibi tekniklerle tespit sistemlerini atlatabilmektedir. Bu nedenle, yeni, dinamik ve öğrenen

sistemlerin geliştirilmesi gerekmektedir. Örneğin www.facebook.com ilk bakışta normal bir siteymiş gibi görünse de .com alan adındaki farklılık kullanıcıyı yanıltmaktadır. Bu gibi basit yöntemler kullanıcıya büyük zararlar verebilmektedir.

Ayrıca mevcut birçok kimlik avı tespit sistemi gerçek zamanlı çalışmamaktadır. Kullanıcı bir bağlantıya tıkladığında analiz süreci geç tamamlandığından, kötü niyetli siteye erişim engellenememekte veya kullanıcı çoktan verilerini paylaşmış olabilmektedir. Bu durum hem bireysel kullanıcılar hem de kurumlar için önemli bir güvenlik açığı oluşturmaktadır. Çünkü sistem ne kadar güvenli ya da öngörülebilir olsa da son kullanıcı davranışları öngörülemezdir ve tehdide en müsait halkadır [14].

Bu bağlamda, gerçek zamanlı olarak URL'leri analiz edebilen, kimlik avı veya zararlı URL'leri anında tespit eden ve kullanıcıyı tarayıcı düzeyinde anında uyararak bir sistem geliştirme ihtiyacı doğurmuştur. Bu tez, bu ihtiyaca yanıt olarak tasarlanmıştır.

1.2. Tezin Amacı ve Kapsamı

Bu tez çalışmasının temel amacı, gerçek zamanlı URL analizi yoluyla kimlik avı saldırılarını yüksek doğrulukla tespit eden ve kullanıcıyı anında bilgilendiren bir sistem geliştirmektir. Bu amaç doğrultusunda hem lexical analiz hem de makine öğrenmesi tabanlı sınıflandırma yöntemleri ve web teknolojileri bir arada kullanılmıştır.

Çalışma kapsamında, etiketlenmiş URL verilerinden oluşan bir veri seti üzerinde analizler gerçekleştirilmiştir. Bu veriler üzerinde lexical özellik çıkarımı uygulanmış; URL uzunluğu, alan adı yapısı, özel karakter dağılımı, karakter tekrarları ve sayısal oranlar gibi URL'nin yapısal özelliklerini temsil eden çeşitli nitelikler elde edilmiştir. Elde edilen bu özellikler, makine öğrenmesi modellerine girdi olarak sunulmuş ve phishing ile legitimate URL'leri ayırt edebilecek bir sınıflandırma modeli geliştirilmiştir.

Modelin eğitimi ve optimizasyonu tamamlandıktan sonra, sistemin gerçek zamanlı olarak tarayıcı üzerinde çalışabilmesi için bir tarayıcı eklentisi (browser extension) geliştirilmiştir. Bu eklenti, kullanıcı bir web sitesine giriş yaptığı anda o URL'yi otomatik olarak modele göndererek analiz eder ve URL zararlı olarak algılandıysa kullanıcıya anında görsel bir uyarı olarak iletir. Böylece sistem, kullanıcı etkileşimi olmadan, arka planda sürekli çalışan bir güvenlik katmanı oluşturur.

Tezin kapsamı, veri toplama, özellik mühendisliği, model eğitimi, performans değerlendirmesi ve gerçek zamanlı uygulama geliştirilmesi aşamalarını içermektedir. Çalışmanın çıktısı, hem akademik düzeyde kimlik avı tespitine katkı sunmayı hem de uygulama düzeyinde kullanılabilir bir güvenlik aracı önermeyi hedeflemektedir.

Sonuç olarak, bu tez çalışması ile siber güvenlik alanında makine öğrenmesi tabanlı, gerçek zamanlı ve kullanıcı dostu bir çözüm geliştirilmiştir. Bu çözüm, kimlik avı saldırılarına karşı proaktif bir savunma yaklaşımı sunarak, geleneksel yöntemlerin ötesine geçmeyi amaçlamaktadır.

1.3. Tez Organizasyonu

Bu tez çalışması 5 ayrı bölümden oluşmaktadır. Birinci bölüm çalışmanın genel hatlarını belirtmiş, kimlik avı saldırıları başta olmak üzere yanıltıcı veya zararlı url adreslerine sahip sitelerin günümüzdeki etkileri, problem tanımı, çalışmanın amacı ve kapsamı genel olarak açıklanmıştır. Bunun yanı sıra geliştirilen sistemin temel hedefleri ve özgün katkıları vurgulanmıştır.

İkinci bölümde, çalışmanın kuramsal temelleri ve literatür çalışması sunulmuştur. Bu kapsamda, web ve web güvenliği kavramları, URL ve URL güvenliği kavramlarının ne olduğu ele alınmıştır. Ardından siber güvenlik alanında kullanılan makine öğrenmesi algoritmalarının ne olduğu, hangi problemlere güvenli çözümler ürettiği ve zararlı URL tespitinde nasıl kullanılabileceği anlatılmıştır. Bölümün son kısmında ise son yıllarda yapılmış güncel akademik çalışmalar incelenmiş ve bu tez çalışmasının literatürdeki boşluğu nasıl doldurduğu tartışılmıştır.

Üçüncü bölümde, URL analizi ve özellik mühendisliği adımları detaylı olarak anlatılmıştır. Bu bölümde eğitilecek model için verilerin nasıl ve nereden toplandığı, bu verilerin ön işleme adımları, lexical (sözcüksel) özelliklerin çıkarımı ve boyut azaltma teknikleri adım adım açıklanmıştır. Ayrıca, modeli eğitmek için kullanılacak özelliklerin seçimi ve bu özelliklerin birbiriyle olan bağlantısını matematiksel olarak gösteren korelasyon hesapları da sunulmuştur.

Dördüncü bölümde, geliştirilen makine öğrenmesi modelleri ve gerçek zamanlı sistem mimarisi anlatılmıştır. Bu kapsamda, kullanılan algoritmalar, model eğitimi, doğrulama ve optimizasyon süreçleri açıklanmış, gerçek zamanlı çalışan tarayıcı eklentisinin tasarımı ve sistem entegrasyonu detaylandırılmıştır. Ayrıca, eklentinin

alışma prensibi, kullanıcı etkileşimi, güvenlik önlemleri ve performans değerlendirme kriterleri bu bölümde ele alınmıştır.

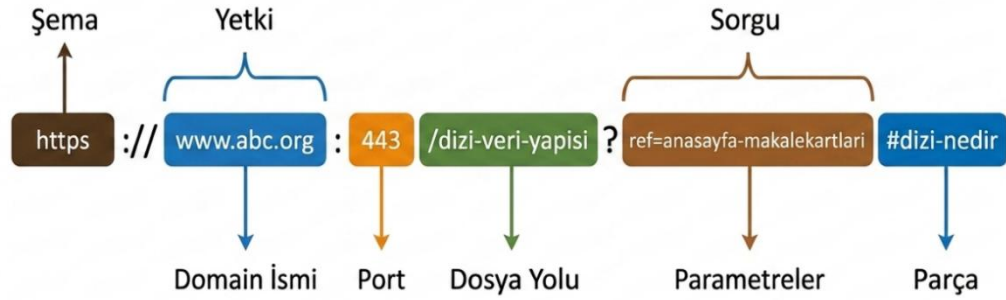
Tezin son bölümünde ise, elde edilen deneysel sonuçlar, modelin performans analizleri sunulmuştur. Elde edilen bulgular yorumlanarak alışmanın literatüre ve uygulamaya sağladığı katkılar tartışılmış, karşılaşılan kısıtlar ve gelecekte yapılabilecek geliştirmelere yönelik öneriler paylaşılmıştır.

Bu yapı sayesinde tez alışması hem teorik temeli hem de pratik uygulamayı bütünleştiren bir perspektifle ele alınmış, kimlik avı saldırılarının gerçek zamanlı tespiti konusunda hem akademik hem de uygulamalı bir özüm önerisi sunulmuştur.

2. TEMEL BİLGİLER VE LİTERATÜR ARAŞTIRMASI

2.1. Web ve URL Güvenliği

Web, günümüzde bilgi paylaşımı, iletişim, ticaret ve hizmet sunumunun temel yapısı haline gelmiştir. Milyonlarca kullanıcının aynı anda çevrimiçi bir eylem gerçekleştirdiği bu sistemde, URL kavramı, web üzerindeki her kaynağın benzersiz olarak tanımlanmasını sağlar. Bir URL, bir web kaynağına erişim için gerekli olan adresleme bilgisini içerir. URL'ler birden fazla farklı şekilde oluşabilir. Şekil 2.1'de örnek bir URL yapısı verilmiştir.



Şekil 2.1. URL bileşenleri

Bir URL, erişilmek istenen kaynağa ilişkin bilgileri bir arada sunan bir adres yapısıdır ve her bileşen belirli bir işlevi yerine getirir. “https” bölümü, istemci ile sunucu arasındaki iletişimde kullanılan şemayı (scheme) ifade eder. www.abc.org kısmı, kaynağın bulunduğu sunucunun alan adı (domain ismi) olup yetki bilgisinin temel bölümünü oluşturur. “:80” ifadesi, sunucunun hangi port üzerinden dinleme yaptığını belirtir. “/dizi-veri-yapisi” bölümü, sunucu üzerindeki dosya yolunu (path) tanımlar ve kaynağın hangi dizinde bulunduğunu gösterir. “?” ile başlayan kısım, sunucuya iletilen sorgu bilgisini (query) temsil eder. Son olarak “#dizi-nedir” bölümü, sayfa içerisindeki belirli bir parçaya (fragment) yönlendirmeyi sağlar. Bu bileşenlerin tamamı birlikte, bir kaynağın nerede bulunduğunu ve nasıl erişileceğini açık bir biçimde tanımlar [15]. URL yapısının bu kadar ayrıntılı olması, aynı zamanda saldırganların bu alanları manipüle ederek kullanıcıları yanıltmasına da imkân tanıdığından, web güvenliği açısından kritik öneme sahiptir.

Web güvenliği, kullanıcıların ve sistemlerin veri gizliliğini, bütünlüğünü ve erişilebilirliğini korumayı amaçlayan bütünsel bir yaklaşımdır [16]. Günümüzde web uygulamaları yalnızca statik içerik sunmakla kalmayıp, dinamik veri akışları, kullanıcı doğrulama sistemleri ve üçüncü taraf API bağlantılarıyla daha karmaşık bir hale gelmiştir. Bu karmaşıklık, beraberinde güvenlik risklerini de getirmektedir.

Web tabanlı tehditler arasında oltalama ya da kimlik avı (phishing), zararlı yazılım (malware) yerleştirme, Cross-Site Scripting (XSS), SQL enjeksiyonu gibi saldırı türleri yer almaktadır. Bu saldırılar genelde kullanıcıları zararlı URL'lere yönlendirmeye başlamakta ve kullanıcıların farkında olmadan bu bağlantılara erişmesini sağlamaktadır. Özellikle kimlik avı saldırılarında, saldırganlar sahte URL'ler oluşturarak kullanıcıların güvenini kötüye kullanmakta ve onları sahte giriş sayfalarına yönlendirmektedir. Bu URL'ler genelde gerçek sitesinin URL'sine çok benzer hazırlanmakta ve ilk bakışta gözle ayırt edilememektedir. Bu şekilde hazırlanan URL'lerin olduğu sitelerin içerikleri de görsel olarak bire bir aynı olmaktadır. Ancak işlevsellik olarak farklı amaçlara hizmet etmektedir.

Bu noktada, URL güvenliği, web güvenliğinin temel bileşenlerinden biri olarak öne çıkar. Güvenli bir URL yapısının sağlanması, kullanıcıların yanıltılmasını ve sistemlerin kötüye kullanılmasını önlemeye yardımcı olur [16]. Örneğin, HTTPS protokolünün kullanımı, verilerin şifrelenmesini sağlayarak "ortadaki adam" (Man-in-the-Middle) saldırılarını büyük ölçüde engeller [16]. Ayrıca alan adı sertifikalarının doğrulanması, kimlik sahteciliği risklerini azaltır. Bununla birlikte, saldırganlar çoğu zaman benzer alan adları, alt alan adları veya link kısaltma servisleri kullanarak bu güvenlik önlemlerini atlatmayı hedefler.

Kurumsal yapılarda web kullanımı ve güvenliği bireysel kullanım ve güvenliğine kıyasla çok daha karmaşık bir yapıya sahiptir. Bireysel kullanıcı doğrudan internete çıkarken kurumsal ağlarda internete çıkmak için arada bir sürü katman ya da cihaz vardır. Kurumsal web altyapıları, genellikle birden fazla sunucu, güvenlik duvarı, DNS hizmeti ve içerik yönetimi sistemi (CMS) barındırır. Bu sistemler üzerinden bir sürü kullanıcı eş zamanlı işlem gerçekleştirir. Dolayısıyla kurumların sadece kullanıcı arayüzünü değil, arka plandaki veri akışını, API entegrasyonlarını ve log yönetim süreçlerini de güvence altına alması gerekir [17].

Kurumsal düzeyde uygulanan bazı güvenlik önlemleri arasında SSL/TLS sertifikaları, Web Application Firewall (WAF) çözümleri, güvenli kimlik doğrulama sistemleri, veri sızıntı önleme (DLP) sistemleri ve saldırı tespit/önleme (IDS/IPS) sistemleri kullanılmaktadır. Ancak tüm bu kullanılan güvenlik çözümleri bile kullanıcı farkındalığı veya bilinci zayıfsa etkisiz hale gelebilir. Çünkü güvenlik sistemlerinde en zayıf halka her zaman insandır. Bu nedenle kullanıcı bilinci veya farkındalığı da kurumsal web güvenliğinin ayrılmaz bir parçasıdır.

Sonuç olarak web ve URL güvenliği yalnızca teknik bir gereklilik değil, güvenli internet kullanımının bir zorunluluğudur. Bu nedenle URL yapılarının analizi ve güvenlik kontrolleri hem bireysel kullanıcılar hem de kurumsal sistemler için internet ortamındaki tehditlerin erken tespiti ve önlenmesi için alınması gereken tedbirler arasında en hızlı ve en kolay yollardan biridir.

2.2. Web Güvenliğinde Makine Öğrenmesi Algoritmaları

Web güvenliğinde ortalama tespit teknolojisi sürekli olarak güncellenmekte ve gelişmektedir. Ortalama bağlantılarını tespit etmek için kullanılan yöntemler kara liste ve kural tabanlı filtreleme yöntemlerine dayanmaktadır. Bu yöntemler önceden bilinen saldırı örüntülerine göre oluşturulmaktadır. Bu nedenle yeni oluşturulan bir URL'nin ortalama web sitesi olup olmadığını doğru ve güvenli bir şekilde tahmin etmemiz gerekir. Yapay zekâ teknolojisinin gelişmesiyle birlikte, tahmin yeteneği, günümüz veri yoğun sistemlerinde ve karar alma süreçlerinde hayati bir beceri haline gelmiştir [10].

Yapay zekâ tabanlı web güvenliği yaklaşımlarında, URL'ye ait verilerden özellik çıkarımı (feature extraction) yapılmakta, ardından bu özellikler bir yapay zekâ modeline girdi olarak verilmekte ve URL'nin zararlı olup olmadığı model tarafından tahmin edilmektedir. Bu alanda yaygın olarak kullanılan algoritmalar şu şekildedir.

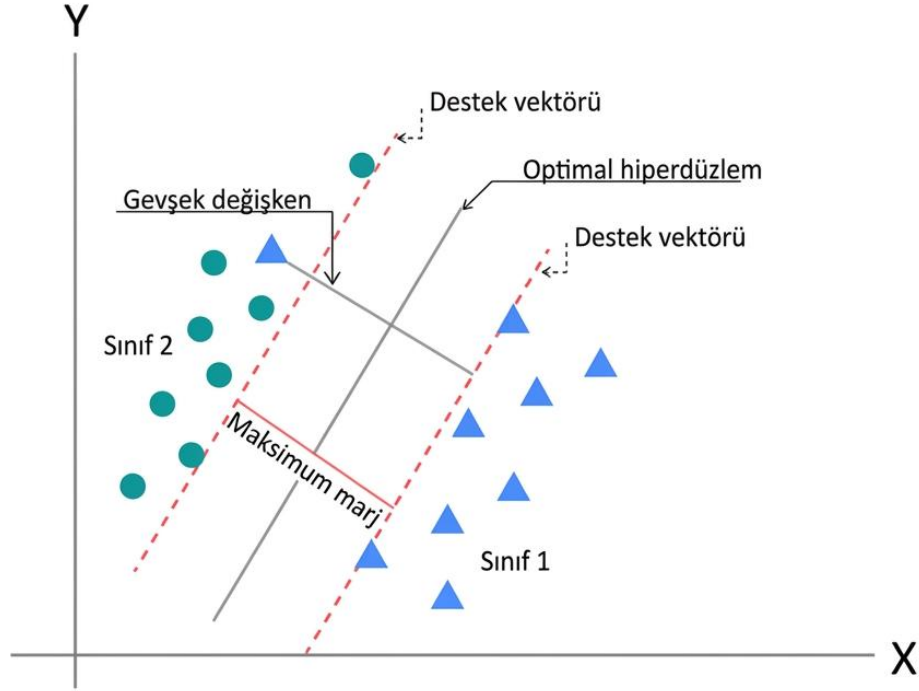
Lojistik regresyon isminde regresyon geçmesine rağmen bir sınıflandırma algoritmasıdır. Bir verinin hangi sınıfa ait olduğunu tahmin eder [4]. Bunu yaparken bir dizi matematiksel formül kullanır ve bu formülün kat sayıları ona verilen veri setinden çıkarılarak oluşturulur [5]. Lojistik regresyon, URL'ye ait özelliklerle (örneğin uzunluk, sembol sayıları, belirli anahtar kelimeler) kimlik avı olma olasılığı arasındaki ilişkiyi modellemekte ve nispeten düşük hesaplama maliyeti sayesinde büyük veri kümelerinde dahi hızlı sonuç üretebilmektedir. Bununla birlikte, doğrusal

karar sınırı varsayımı nedeniyle karmaşık ve çok boyutlu örüntüleri yakalamakta sınırlı kalabilmekte, bu durumda daha esnek algoritmalarla birlikte karşılaştırmalı olarak kullanılmaktadır [7].

Naive Bayes algoritması, özellikler arasında koşullu bağımsızlık varsayımına dayanan temel bir makine öğrenmesi algoritmasıdır. Bu algoritma, özellikle metin veya URL'ler gibi yüksek boyutlu özellik uzaylarında hızlı ve hafif çözümlerle öne çıkar. Algoritma, URL'lerdeki karakter dizileri, token'lar veya belirli niteliklerin varlık/yokluk bilgisi üzerinden olasılıksal hesaplamalar yaparak sınıflandırma kararları üretir, bu sayede düşük hesaplama maliyetli ve kolay uygulanabilir olması nedeniyle çoğu çalışmada temel karşılaştırma modeli olarak kullanılabilir. Ancak, verilerde karmaşık ilişkilerin veya korelasyonların bulunduğu senaryolarda performansı, daha gelişmiş öğrenme modellerinin gerisinde kalabilmektedir [7].

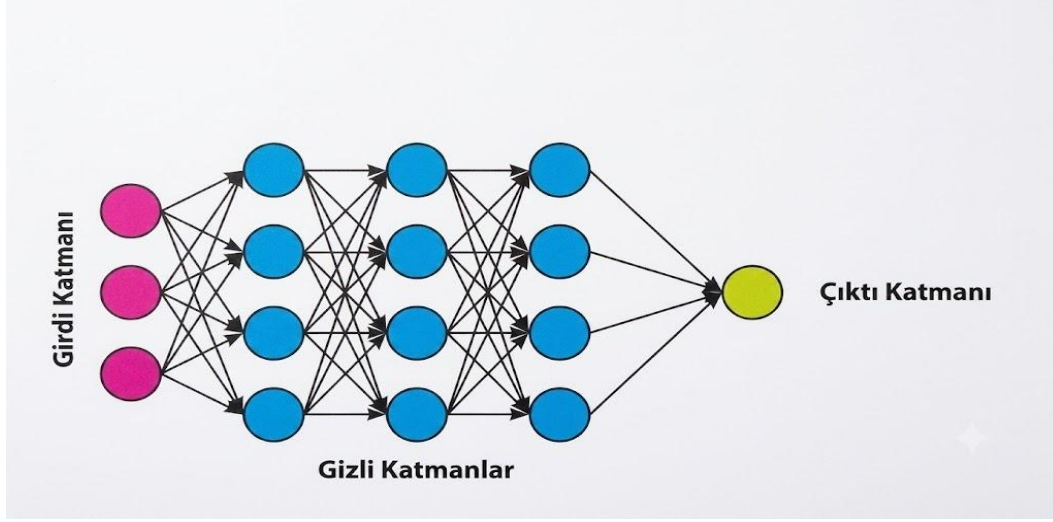
Random Forest, çok sayıda karar ağacının bir araya gelerek çoğunluk oyu ile karar verdiği bir toplu modelleme (küme) yöntemidir ve URL temelli phishing tespitinde literatürde en başarılı algoritmalarından biri olarak görülmektedir [7,8,9]. Farklı alt örneklem kümeleri ve özellik alt uzayları üzerinden eğitilen ağaçların bir araya gelmesi, modelin hem genelleme yeteneğini artırmakta hem de tekil ağaçlara göre aşırı uyum riskini azaltmaktadır. Birçok çalışmada, lexical ve host tabanlı URL özellikleri kullanılarak Random Forest algoritmasıyla %95–99 aralığında doğruluk oranlarına ulaşıldığı görülmüştür [7-9].

Destek vektör makineleri (SVM) sınıflandırma problemlerinde yaygın olarak kullanılmaktadır [6]. Bir URL'nin zararlı mı yoksa zararsız mı olduğu konusu bu tip bir sınıflandırma problemi olduğu için bu alanda da birçok örnek kullanım mevcuttur. SVM, Şekil 2.2'de gösterildiği gibi sınıflar arasındaki marjini maksimize eden bir karar sınırı öğrenerek, doğru ayarlanmış çekirdek (kernel) fonksiyonları ile karmaşık ayırım yüzeyleri oluşturabilmektedir. Literatürde, RBF çekirdekli SVM modelleri ile URL temelli kimlik avı tespitinde yüksek doğruluk ve düşük yanlış pozitif oranları elde edildiği rapor edilmiştir [11]. Ancak SVM'nin büyük veri kümelerinde eğitim maliyeti yüksek olabilmekte ve gerçek zamanlı güncelleme gerektiren senaryolarda kullanımı sınırlanabilmektedir. Aşağıdaki şekilde SVM'nin genel yapısı gösterilmektedir.



Şekil 2.2. Destek vektör makinelerinde optimal ayırım düzlemi ve destek vektörleri [17]

Bu alanda kullanılan diğer bir algoritma da yapay sinir ağlarıdır. Yapay sinir ağları, insan beyninin çok katmanlı yapısından ve bilgiyi işleme yapısından esinlenilerek oluşturulmuştur [18]. İnsan beyninde birbirlerine sinapslarla bağlı birçok sinir hücresi bulunmaktadır. Benzer şekilde yapay sinir ağlarında da yapay sinapslarla birbirine bağlı çok sayıda yapay nöronlar bulunur [18]. Sinir ağları bu yapısından dolayı, özellikle karmaşık, doğrusal olmayan ilişkilere sahip veri setlerini modellemede başarılıdır. URL'ye ait lexical ve host tabanlı özellikler bir veri seti halinde ağa verilerek her katmanda gizli nöronlar aracılığıyla soyut temsil öğrenilir ve çıktı katmanında zararlı zararsız kararı verilir [7]. Bununla birlikte, makine öğrenmesi tabanlı yaklaşımlar çoğunlukla manuel özellik mühendisliğine dayandığından, son yıllarda özellik çıkarımını kendisi yapabilen derin öğrenme mimarilerine olan ilgi artmıştır. Şekil 2.3'te yapay sinir ağlarının örnek mimarisi verilmiştir.



Şekil 2.3. Yapay sinir ağları [19]

Son yıllarda, web güvenliğinde hibrit ve toplu modelleme temelli yaklaşımlar, yani birden fazla makine öğrenmesi veya derin öğrenme algoritmasının aynı anda kullanıldığı sistemler yaygınlaşmıştır. Son yıllarda, web güvenliği alanında hibrit ve toplu modelleme temelli yaklaşımlar, yani birden fazla makine öğrenmesi veya derin öğrenme algoritmasının bir arada kullanıldığı sistemler de yaygınlaşmıştır [20]. Bu çalışmalarda, genellikle URL dizilerinden derin öğrenme ile otomatik özellik çıkarımı yapılmakta, ardından gradyan artırma, Random Forest gibi güçlü sınıflandırıcılarla nihai karar üretilmekte veya birden fazla modelin çıktıları birleştirilerek (stacking, voting) daha kararlı sonuçlar elde edilmektedir [20]. Ayrıca, eğitilen bu modellerin tarayıcı eklentileri, istemci ajanları veya web proxy sistemleri ile bütünleştirilerek gerçek zamanlı phishing tespitinde kullanıldığı çalışmalar da mevcuttur [14].

Özet olarak, web güvenliğinde ve özellikle URL temelli kimlik avı tespitinde lojistik regresyon ve Naive Bayes gibi temel sınıflandırıcılardan, Random Forest ve gradyan artırma gibi güçlü toplu modelleme yöntemlerine SVM'den derin öğrenme tabanlı CNN ve LSTM modellerine kadar geniş bir algoritma yelpazesi kullanılmaktadır. Bu tez çalışmasında, URL'lerin lexical özelliklerine dayalı makine öğrenmesi modelleri kullanılarak zararlı URL'lerin gerçek zamanlı tespiti amaçlanmakta, bir sonraki bölümde (2.3. Lexical Analiz) bu modellerin girdisi olan sözcüksel özelliklerin nasıl elde edildiği ayrıntılı olarak ele alınmaktadır.

2.3. Lexical Analiz

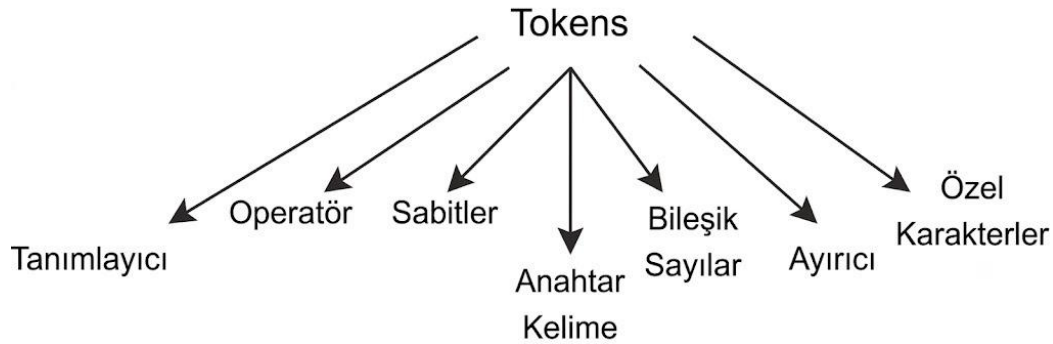
Lexical analiz, genel anlamda bir metni veya karakter dizisini daha küçük anlamlı birimlere (token) ayırarak bu birimler üzerinden yapısal ve istatistiksel özellikler elde etmeyi amaçlayan bir metin işleme tekniğidir. Şekil 2.4'te token özellikleri gösterilmiştir. Derleyici tasarımında kaynak kodun anahtar kelimeler, operatörler ve semboller gibi bileşenlere ayrılması için kullanılan bu yaklaşım, günümüzde doğal dil işleme, bilgi erişimi ve güvenlik alanlarında da yaygın biçimde kullanılmaktadır. Temel amaç, ham karakter dizisini doğrudan kullanmak yerine ondan uzunluk, frekans, sembol kullanımı ve belirli desenlerin varlığı gibi öznitelikler çıkararak daha anlamlı bir temsil oluşturmaktır [21].

URL tabanlı zararlı bağlantı tespitinde lexical analiz, URL'yi bir metin gibi ele alır ve doğrudan bu metin üzerinde çalışır. Bu durum, özellikle gerçek zamanlı sistemler için hesaplamada düşük maliyet ve düşük gecikmeye sahiptir. URL'nin genel yapısında yer alan host, path ve query bileşenleri, belirli ayırıcı karakterlere göre parçalanarak tokenlerine ayrılır ve bu tokenler üzerinde çeşitli matematiksel özellikler üretilir [11, 12].

Lexical analizde kullanılan başlıca özellik gruplarından ilki, uzunluk ve karakter temelli özelliklerdir. Toplam URL uzunluğu, host ve path uzunlukları, URL içindeki özel karakter sayıları (“@”, “-”, “_”, “%”, “=”, “&”, “?” vb.) ve bu karakterlerin oranları, kimlik avı URL'lerinin karmaşık ve maskeleyici yapısını ortaya çıkarmada sıkça kullanılan ve temel olan özelliklerdir. Bir diğer önemli grup ise alt alan adı yapısı ve nokta sayısı gibi domain özellikleridir. Zararsız siteler çoğu zaman daha sade alan adları kullanırken, zararlı siteler uzun ve iç içe geçmiş alt alan adları, fazla nokta sayısı ve belli uzantıların (.xyz vb.) yoğun kullanımı görülebilmektedir. Ayrıca bu uzantıların türü ve belirli uzantıların zararlı URL ile ilişkilendirme oranı lexical analizde değerlendirilen özellikler arasındadır [7, 11].

Lexical analizde ayrıca URL içerisinde yer alan anahtar kelimelerin (keywords) tespiti önemli yer tutar. “login”, “secure”, “verify”, “update”, “account”, “bank” gibi kelimeler, kullanıcıların güvenini ve merakını suistimal etmeye yönelik olarak zararlı URL'lerde sıkça kullanılmaktadır. Bu nedenle belirli anahtar kelimelerin var/yok bilgisi, sayısı veya kombinasyonları, URL'nin kötü niyetli olma olasılığını artıran özellikler olarak modele dahil edilir. Ek olarak, sayısal karakter oranı, büyük/küçük harf oranı, harf-sayı karışık dizilerin uzunluğu gibi oran temelli özellikler, doğal

görünen alan adları ile otomatik üretilmiş veya gizleme amaçlı tasarlanmış adresleri ayırt etmede etkili olmaktadır.



Şekil 2.4. Token özellikleri [21]

2.4. Literatür Çalışmaları

Tang ve Mahmoud (2021), kimlik avı tespitinde kullanılan makine öğrenmesi yöntemlerini kapsamlı biçimde incelemiş ve özellikle URL'lerin sözcüksel özelliklerinin tespit başarısını önemli ölçüde artırdığını belirtmiştir [10]. Araştırmacılar, lexical özelliklerin hızlı çıkarılabilir olması sayesinde gerçek zamanlı sistemlerde önemli bir avantaj sunduğunu vurgulamışlardır. Çalışmada SVM, Random Forest ve lojistik regresyon gibi farklı algoritmalar karşılaştırılmış ve bu yöntemlerin çeşitli veri setlerinde yüksek doğruluk oranlarına ulaştığı gösterilmiştir. Yazarlar, phishing saldırılarının giderek daha karmaşık hâle geldiğini, bu nedenle kara liste yaklaşımlarının artık yeterli olmadığını ifade etmiştir. Ayrıca gerçek zamanlı tespit altyapılarının hâlâ sınırlı kaldığı belirtilmiş ve dinamik, güncel veri gereksinimine dikkat çekilmiştir. Bu bağlamda çalışma, hızlı ve çevik tespit yöntemlerinin geliştirilmesinin önemini vurgulamaktadır.

Geyik (2021), URL tabanlı kimlik avı saldırılarının tespitinde farklı makine öğrenmesi algoritmalarını karşılaştırmış ve özellikle lexical özelliklerin sınıflandırma performansı üzerindeki etkisini ortaya koymuştur [7]. Yapılan çalışmada URL uzunluğu, karakter çeşitliliği ve alt alan adı seviyesi gibi temel özellikler özellik mühendisliği yöntemleriyle analiz edilmiş ve model eğitiminde kullanılmıştır. SVM ve Random Forest algoritmalarının yüksek doğruluk oranı sunduğu belirtilmiştir. Yapılan çalışmada, yanlış sınıflandırmaların özellikle sınır değerdeki URL'lerde ortaya çıktığını ve bunun gerçek zamanlı sistemlerde önemli bir sorun oluşturduğunu vurgulanmıştır. Ayrıca veri setinin güncellik durumunun modele doğrudan etki ettiği

ifade edilmiştir. Çalışma, phishing URL'lerinin sürekli değiştiğini ve modellerin kendi başına bırakılmaması gerektiğini göstermiştir.

Luhach ve arkadaşları (2021), zararlı URL tespitinde makine öğrenmesi tabanlı hibrit bir yaklaşım sunmuş ve lexical, host tabanlı ve içerik tabanlı özelliklerin bir arada kullanılmasının performansı artırdığını göstermiştir [8]. Lexical özelliklerin düşük maliyetli ve hızlı hesaplanabilir olması sebebiyle gerçek zamanlı analizlerde kritik role sahip olduğu vurgulanmıştır. Araştırmada toplu modelleme yöntemlerinin daha stabil sonuçlar verdiği belirtilmiş ve özellikle Gradient Boosting yönteminin karmaşık URL örüntülerini başarıyla yakaladığı ifade edilmiştir. Phishing URL'lerinin belirli yapısal benzerlikler taşımasından dolayı sözcüksel analiz yaklaşımının güçlü bir performans sunduğunu ifade etmiştir. Ayrıca çalışma, gerçek zamanlı filtreleme sistemleri için çeşitli mimari öneriler sunarak bu alandaki eksikliklere dikkat çekmiştir.

Alzboon ve Abualigah (2025), makine öğrenmesi yöntemleriyle phishing web sitelerinin tespiti üzerine gerçekleştirdikleri çalışmada hem lexical hem de davranışsal özellikleri kullanarak model performansını artırmayı hedeflemiştir [9]. Araştırmacılar, derin öğrenme yöntemlerinin karmaşık örüntüleri daha etkili bir şekilde yakaladığını ve geleneksel yöntemlere göre daha başarılı olduğunu belirtmiştir. Yanlış pozitif oranlarının düşürülmesi için özellik seçiminin kritik önemde olduğu ifade edilmiştir. Çalışma, phishing teknolojilerinin giderek daha gelişmiş hâle geldiğini ve bu nedenle klasik yaklaşımların etkinliğini kaybettiğini ortaya koymuştur. Ayrıca gerçek zamanlı sistemlerde hız ve optimizasyon gereksiniminin arttığına değinmiştir. Bu çalışma, gelecekte daha esnek modellerin kullanılmasının önemine işaret etmektedir.

2025 yılında yayımlanan “Phishing URL Detection Using Deep Learning: A Resilient Approach to Mitigating Emerging Cybersecurity Threats” çalışması, derin öğrenme yöntemlerinin manuel öznitelik mühendisliğine gerek duymadan URL sınıflandırmasında sunduğu avantajları incelemiştir [18]. Araştırmada, yerel özellikleri çıkaran paralel CNN katmanları ile ardışık bağımlılıkları yakalayan BiGRU mimarisi birleştirilmiş ve karakter düzeyinde gömme (character-level embedding) kullanımı sayesinde %98,46 gibi yüksek doğruluk oranlarına ulaşıldığı belirtilmiştir. Özellikle ham URL metninden otomatik özellik çıkarabilme kapasitesi sayesinde, modelin sıfırdan gün (zero-day) saldırılarını ve karmaşık gizleme tekniklerini tespit etmede oldukça başarılı olduğu ortaya konmuştur. Bununla birlikte, hibrit modellerin hesaplama maliyetlerinin gerçek zamanlı uygulamalar için bir zorluk teşkil edebileceği

ancak önerilen modelin 1,31 MB gibi düşük bir bellek gereksinimiyle kaynak verimliliği sağladığına dikkat çekilmiştir. Çalışma, ortalama tespit kabiliyetini geliştirmek için gelecekte Transformer tabanlı (BERT veya GPT gibi) mimarilerin entegre edilmesi gerektiğini vurgulamıştır. Bu yönüyle araştırma, ortalama saldırısı tespitinde derin öğrenme tabanlı uçtan uca sistemlerin siber savunmada standart ve ölçeklenebilir bir çözüm haline gelebileceğini göstermektedir.

Rehman ve arkadaşları (2025), gerçek zamanlı phishing tespiti üzerine yaptıkları çalışmalarında makine öğrenmesi modellerinin çevrim içi analizlerde nasıl daha hızlı ve etkili çalışabileceğini incelemiştir [14]. Çalışmada, bağlantıya tıklama anında karar verebilen sistemlerin kritik önemde olduğu belirtilmiş ve çeşitli hafif sınıflandırma algoritmaları karşılaştırılmıştır. Logistic Regression ve SVM gibi düşük hesaplama maliyetli algoritmaların gerçek zamanlı tespit için daha uygun olduğu ifade edilmiştir. Geleneksel kara liste yöntemlerinin dinamik tehdit ortamında etkisiz kaldığı vurgulanmış ve esnek sistemlerin gerekliliği ortaya konmuştur. Model performansının veri güncelliği ile doğrudan ilişkili olduğu belirtilmiştir. Bu çalışma, hızlı tepki veren tespit sistemlerinin güvenlik ekosisteminde temel bir gereksinim olduğunu göstermektedir.

Raja (2021), kimlik avı URL tespitinde sözcüksel özelliklerin sınıflandırma performansı üzerindeki etkisini inceleyen önemli bir çalışma gerçekleştirmiştir [12]. Araştırmada URL yapısında yer alan uzunluk, alt alan adlarının sayısı, özel karakterlerin kullanımı ve belirli anahtar kelimelerin varlığı gibi birçok parametre değerlendirilmiştir. Karar ağaçları ve SVM gibi algoritmalar test edilmiş ve lexical özelliklerin doğru seçildiğinde yüksek performans sunduğu gösterilmiştir. Yazar, phishing URL'lerinin önemli bir bölümünün benzer yapısal şablonlar içerdiğini ve bu nedenle sözcüksel analiz yaklaşımının hâlen etkili olduğunu vurgulamıştır. Ayrıca veri setlerinin güncel olmaması durumunda modellerin hızla performans kaybettiği belirtilmiştir. Çalışma, dinamik tehdit ortamında sürekli güncellenen modellere duyulan ihtiyacı ortaya koymaktadır.

Literatürde bulunan diğer çalışmalar kullanılan algoritmalar ve başarı oranları Tablo 2.1'de paylaşılmıştır. Literatürdeki bu çalışmalar incelendiğinde, makine öğrenmesi ve derin öğrenme tabanlı yöntemlerin ortalama saldırılarının tespitinde önemli ilerlemeler kaydettiği görülmektedir. Ancak gerçek zamanlı analiz kapasitesi, düşük gecikme süresi, veri setlerinin güncelliği ve kullanıcı davranışlarının öngörülemezliği

gibi kritik noktalarda hâlâ önemli eksiklikler bulunmaktadır. Özellikle mevcut yöntemlerin büyük kısmı offline analizlere dayanmakta, gerçek zamanlı karar verme sürecinde gecikmeler yaşamakta ve sıfıncı gün saldırılarını her zaman tespit edememektedir. Bu çalışma, söz konusu eksiklikleri gidermek amacıyla URL'nin lexical özelliklerini kullanarak hızlı, hafif ve gerçek zamanlı çalışabilen bir makine öğrenmesi modeli geliştirmeyi hedeflemektedir. Böylece hem yeni ortaya çıkan phishing saldırılarına karşı etkili hem de kullanıcı deneyimini kesintiye uğratmayan bir sistem tasarlanması amaçlanmaktadır.

Tablo 2.1. Literatür çalışmaları

Reference	Year	Classifier / Method	Result
[22]	2021	Attention-based bidirectional independent recurrent network (Bi-IndRNN), and capsule network (CapsNet)	99.89%
[23]	2019	Random Forest, Gradient Boost, AdaBoost, Logistic Regression, Naive Bayes	92%, 90%, 90%, 87%, 70%
[24]	2023	SVM, Random Forest, Decision Tree, K-Nearest Neighbors (KNN)	91.75%, 92.15%, 90.18%, 86.64%
[25]	2022	CNN, LSTM, Naive Bayes, Random Forest	95.13%, 95.14%, 96.01%, 95.15%
[26]	2021	XGBoost, CS-XGBoost, SMOTE+XGBoost	97.83%, 99.05%, 98.43%
[27]	2023	DDQN-based classifier, Deep Reinforcement Learning	93.4%
[28]	2023	Random Forest, LightGBM, XGBoost	96.6%, 95.6%, 93.2%
[29]	2020	Random Forest, SVM	99.77%, 93.39%
[30]	2019	Random Forest, SVM	92.38%, 87.93%
[31]	2023	MM-ComBERT-LMS	98.72%
[32]	2023	Naive Bayes, CNN, Random Forest, LSTM	96%
[33]	2022	Random Forest	96%
[34]	2022	MLP	99.62%
[35]	2022	BERT, NLP, Deep CNN	96.6%
[36]	2023	Random Forest, Gradient Boost, XGBoost	97.44%, 98.27%, 98.21%
[37]	2021	Classification Based on Association (CBA)	91.30%
[38]	2022	LSTM, Bi-LSTM, GRU	97%, 99%, 97.5%
[39]	2021	XGBoost, CS-XGBoost, SMOTE+XGBoost	99.8%

Tablo 2.1. (Devamı) Literatür çalışmaları

Reference	Year	Classifier / Method	Result
[40]	2021	Logistic Regression,	97.5%
		Decision Tree	85%
[41]	2020	Random forest,	86.24%
[42]	2020	Single class SVM	96-97%
		Random forest,	96.99%
		fast.ai,	97.55%
		Keras-TensorFlow(deep learning framework)	93.81%
[43]	2022	Logistic Regression,	93.26%
[44]	2017	MLP neural network	96.35%
		Multi-layer filtering model,	79.55%
		Naive Bayes,	77.30%
[45]	2022	Decision Tree,	79.35%
		SVM	76.80%
		Logistic regression,	92.80%
		SVM,	97.32%
		Random forest,	97.35%
		Gradient Boost,	96.27%
		Bagging	97.35%
[46]	2019	CNN	86.63%
[47]	2022	Logistic regression,	89.99%
		SVM,	96.49%
		ELM,	98.17%
		ANN	97.20%
[48]	2021	1D-CNN,	~ 99%
[49]	2022	LSTM	
		Random forest	99.8%
[50]	2017	Expose neural network that uses deep learning method	97-99%
[51]	2012	Logistic regression,	87.67%
		support vector classification (SVC)	86%
[52]	2022	Decision Tree,	96.33%
		Random forest	97.49%
[53]	2016	Auto-updated whitelist	89.38%
[54]	2020	AdaBoost-ExtraTree (ABET),	97.485%,
		Bagging –Extra tree (BET)	97.404%,
		Rotation Forest – Extra Tree (RoFBET),	97.449%,
		LogitBoost-Extra Tree (LBET)	97. 576%
[55]	2021	LSTM,	79.5%,
		DCNN,	80.6%,
		CNN-LSTM,	91.4%,
		DTCNN-LSTM	93.2%
[56]	2021	Feature selection based on chi-square, Pearson correlation, and score	99.93%
[57]	2018	SVM,	96.52%
		MLP,	93.48%
		BN,	88.99%
		Logistic Regression	86.16%
[58]	2017	Naive Bayes,	95.06%
		J48,	99.22%
		SVM,	94.55%
		KNN	97.14%
[59]	2019	N-gram	94.04%
[60]	2020	KNN (without entropy)	99.2%
[61]	2020	Logistic Regression,	95.13%,
		SVM,	95.34%,
		CNN,	96.87%,
		DBN-SVM	99.96%
[62]	2024	Random Forest	92.86%
		Support Vector Machine	93.33%
		Naive Bayes	78.53%
[63]	2024	Random Forest	96.4%
		Naive Bayes	81.6%
		Logistic Regression	83.9%
		CNN	94.7%

Tablo 2.1. (Devamı) Literatür çalışmaları

Reference	Year	Classifier / Method	Result
[64]	2024	Random Forest	91.93%
		Decision Tree	90.00%
		Logistic Regression	66.48%
		XGBoost	87.23%
		Naive Bayes	31.46%
		K-Nearest Neighbors	87.52%
		Gradient Boosting	81.39%
[65]	2024	Extra Trees	92.01%
		Decision Tree	99.36%
		Random Forest	99.61%
		Multilayer Perceptron	99.27%
[66]	2025	Tiny Llama + DL	96.55
		BERT (Base) + DL	97.5%
		T5 (Large) + DL	97.5%
		GPT-2 + DL	97.0%
[14]	2025	KNN	99.77%
		Naive Bayes	99.96%
		Naive Bayes Kernel	99.97%
		Random Forest	99.99%
		Random Tree	95.38%
		Decision Tree	99.99%
		LGM	96%
[67]	2025	XGBoost	96%
		Gradient Boosting	94%
		Random Forest	97%
		LSTM	95%
		CNN	95%
		CNN+LSTM	95.6%
		GA+CNN+LSTM	92%
		PSO+CNN+LSTM	81%
		HHO+CNN+LSTM	95.7%
		ELECTRA	99%
[68]	2025	Logistic Regression	77%
		Random Forest	81%
		Gradient Boost	80%
		Hybrid (Proposed)	84%
[69]	2026	SVM	94%
		XGBoost	95%
		Naive Bayes	67%
		Random Forest	97%
		Logistic Regression	94%
		LightGBM	96%
		CatBoost	95%

3. URL ANALİZİ VE ÖZELLİK MÜHENDİSLİĞİ

3.1. URL Analizi Süreci ve Genel Yapı

Bu çalışmada, phishing saldırılarının tespiti amacıyla URL tabanlı bir analiz yaklaşımı geliştirilmiştir. Önerilen sistem, ham URL verilerinin işlenmesinden başlayarak anlamlı özelliklerin çıkarılması, bu özelliklerin seçilmesi ve makine öğrenmesi modellerine girdi olarak sunulmasına kadar uzanan çok aşamalı bir işlem hattından (pipeline) oluşmaktadır.

Çalışma kapsamında, StealthPhisher 2025[70] veri seti kullanılmıştır. Bu veri seti, phishing ve legitimate URL örneklerini içeren dengeli bir yapıya sahiptir ve toplamda 336.750 URL kaydından oluşmaktadır. Dengeli veri yapısı, modelin her iki sınıfı da etkin şekilde öğrenmesini desteklemektedir.

URL analizi süreci veri ön işleme, özellik çıkarımı (feature extraction), özellik seçimi (feature selection) ve model girdisi oluşturma aşamalarından oluşmaktadır. İlk aşamada, ham URL verileri analiz edilebilir bir formata dönüştürülmüş ve gerekli ön işleme işlemleri uygulanmıştır.

Özellik çıkarımı aşamasında, yalnızca URL metni kullanılarak lexical özellikler elde edilmiştir. Bu özellikler, literatürde yer alan çalışmalar incelenerek belirlenmiş ve veri集中的 sadece url den çıkarılabilecek özellikleri bırakıp, ek bir bilgi kaynağı veya işlem gerektirecek öznitelikler silinerek oluşturulmuştur. Bu kapsamda 54 farklı özellik tanımlanmış ve bu özellikler Python programlama dili kullanılarak URL'lerden otomatik olarak çıkarılmıştır. Elde edilen özellikler URL uzunluğu, özel karakter sayıları, alt alan adı bilgileri, karakter dağılımları, entropi ve karmaşıklık ölçütleri gibi URL'nin yapısal özelliklerini temsil etmektedir.

Başlangıçta oluşturulan geniş özellik kümesi, yüksek boyutluluk ve bazı özellikler arasındaki güçlü ilişkiler nedeniyle doğrudan modele verilmemiştir. Bu nedenle, özellik seçimi aşamasında korelasyon analizi ve Mutual Information yöntemleri birlikte kullanılmıştır. Bu süreçte, birbirini tekrar eden veya modele düşük katkı

sağlayan özellikler elenmiş ve daha anlamlı, kompakt bir özellik alt kümesi elde edilmiştir.

Son aşamada, seçilen özellikler sayısal vektörler halinde düzenlenerek makine öğrenmesi modellerine girdi olarak sunulmuştur. Bu yaklaşım, modelin daha verimli öğrenmesini sağlamış, hesaplama maliyetini azaltmış ve aşırı öğrenme (overfitting) riskini düşürmüştür.

Sonuç olarak, önerilen URL analizi süreci, yalnızca URL verisinden elde edilen özellikler ile phishing tespitinde etkili ve uygulanabilir bir yöntem sunmaktadır.

3.2. Veri Kaynakları ve Toplama Yöntemi

Bu çalışmada kullanılan veri seti, phishing ve legitimate URL örneklerini içeren StealthPhisher 2025 [70] veri setidir. Söz konusu veri seti, gerçek dünya senaryolarını yansıtacak şekilde farklı türde URL örneklerini içermekte ve makine öğrenmesi tabanlı analizler için uygun bir yapı sunmaktadır.

Veri seti toplamda 336.750 URL kaydından oluşmakta olup, phishing ve legitimate sınıflar dengeli bir dağılıma sahiptir. Dengeli veri yapısı, modelin her iki sınıfı da tarafsız bir şekilde öğrenmesini sağlamakta ve sınıf dengesizliğinden kaynaklanabilecek yanlılıkları azaltmaktadır.

Çalışma kapsamında herhangi bir ek veri toplama süreci yürütülmemiş, mevcut veri seti doğrudan analiz edilmiştir. Bu yaklaşım, modelin performansının standart ve tekrar üretilebilir (reproducible) bir veri kaynağı üzerinde değerlendirilmesini sağlamaktadır.

Veri seti, URL'lerden türetilmiş çeşitli özellikleri içerecek şekilde yapılandırılmıştır. Ancak bu çalışmada, yalnızca URL yapısından doğrudan elde edilebilen lexical özellikler dikkate alınmış harici kaynaklara dayalı (örneğin WHOIS, DNS veya içerik analizi gerektiren) nitelikler veri setinden çıkarılmıştır. Buna ek olarak, literatürde yer alan çalışmalar incelenerek belirlenen ve URL üzerinden hesaplanabilen yeni özellikler Python tabanlı yöntemlerle türetilerek veri setine dahil edilmiştir. Bu yaklaşım sayesinde, tamamen URL tabanlı, hızlı ve düşük maliyetli bir analiz süreci oluşturulmuştur.

Sonuç olarak, kullanılan veri seti ve veri toplama yaklaşımı, çalışmanın gerçek zamanlı uygulanabilirliğini destekleyen, sade ve etkili bir veri temeli sunmaktadır.

3.3. Veri Ön İşleme Aşamaları

Veri ön işleme aşaması, ham veri setinin analiz ve modelleme sürecine uygun hale getirilmesi açısından kritik bir adımdır. Bu çalışmada, kullanılan veri seti başlangıçta belirli özellikleri içerecek şekilde yapılandırılmış olsa da, model performansını artırmak ve veri tutarlılığını sağlamak amacıyla çeşitli ön işleme adımları uygulanmıştır.

İlk olarak, veri seti üzerinde gereksiz veya çalışmanın kapsamı dışında kalan özellikler tespit edilerek veri setinden çıkarılmıştır. Özellikle URL yapısından doğrudan elde edilemeyen ve harici kaynaklara dayanan özellikler elenerek analiz süreci sadeleştirilmiştir. Bu sayede, modelin yalnızca URL tabanlı nitelikler üzerinden öğrenmesi sağlanmıştır.

Ardından, veri setinde yer alan eksik veya hatalı kayıtlar kontrol edilmiş ve veri bütünlüğünü bozabilecek örnekler temizlenmiştir. Bu işlem, modelin hatalı öğrenme gerçekleştirmesini engellemek açısından önemlidir.

Bir sonraki adımda, özelliklerin veri tipleri ve formatları düzenlenmiş, tüm değişkenlerin makine öğrenmesi algoritmalarına uygun sayısal formata dönüştürülmesi sağlanmıştır. Ayrıca, bazı oran tabanlı özelliklerde ortaya çıkabilecek tanımsız değerler (örneğin sıfıra bölme durumu) kontrol edilerek gerekli düzeltmeler yapılmıştır.

Son olarak, veri seti model eğitime uygun hale getirilmiş ve özellik mühendisliği ile elde edilen niteliklerin tutarlı bir yapı içerisinde kullanılabilmesi sağlanmıştır. Bu aşama, sonraki süreçlerde gerçekleştirilecek özellik seçimi ve model eğitimi adımlarının sağlıklı bir şekilde yürütülmesine zemin hazırlamaktadır.

3.4. Lexical Özelliklerin Belirlenmesi

Bu çalışmada, phishing URL'lerin tespit edilmesi amacıyla yalnızca URL metni üzerinden elde edilebilen lexical (metinsel) özellikler kullanılmıştır. Lexical özellikler, URL'nin yapısal ve karakteristik özelliklerini temsil eden ve harici veri kaynaklarına ihtiyaç duymadan doğrudan hesaplanabilen niteliklerdir.

Özellik belirleme sürecinde, literatürde yer alan çalışmalar incelenmiş ve phishing URL'lerin ayırt edilmesinde etkili olduğu belirlenen özellikler dikkate alınmıştır. Bununla birlikte, veri setinden elde edilebilecek ek özellikler de değerlendirilerek

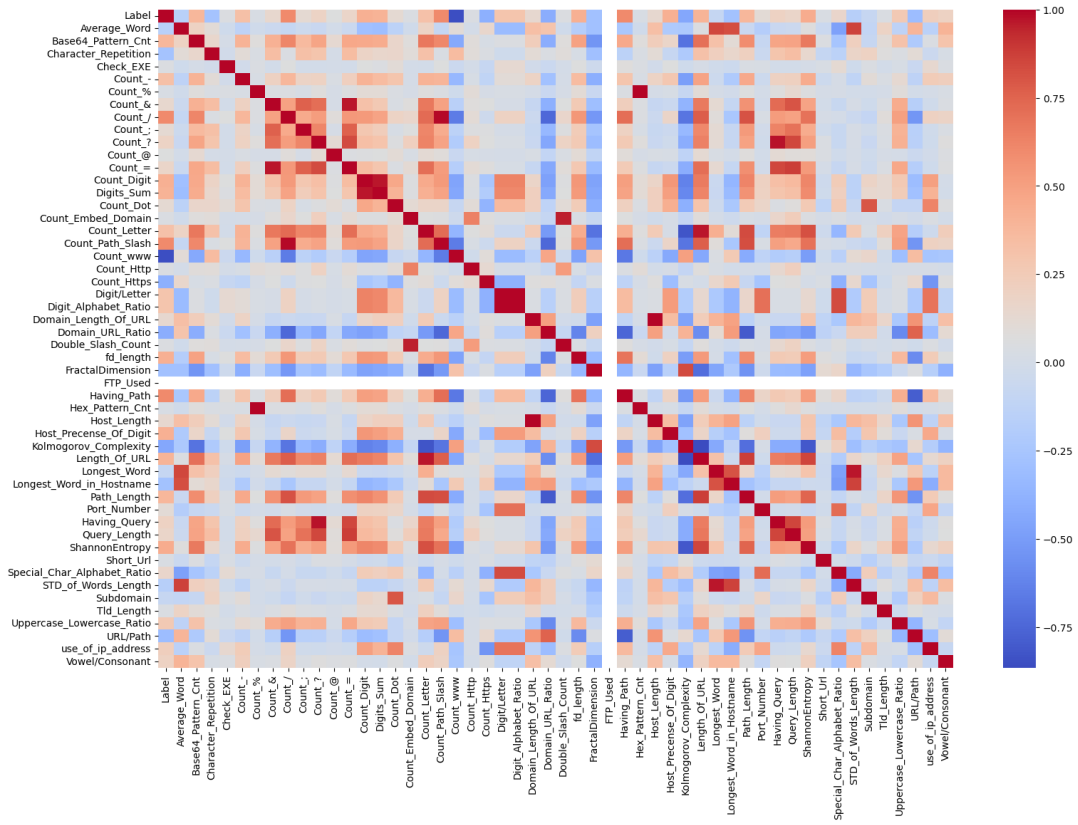
kapsamlı bir özellik kümesi oluşturulmuştur. Bu doğrultuda 54 farklı özellik Python programlama dili kullanılarak URL’lerden otomatik olarak çıkarılmıştır.

Elde edilen lexical özellikler, URL’lerin yapısal, istatistiksel ve karakter dağılımına dayalı özelliklerini temsil edecek şekilde oluşturulmuştur. Bu özellikler URL uzunluğu, alan adı yapısı, özel karakter kullanımı, karakter dağılım oranları, karmaşıklık ölçütleri ve çeşitli desen analizlerini kapsamaktadır. Böylece phishing ve legitimate URL’ler arasındaki yapısal farklılıkların sayısal olarak ifade edilmesi sağlanmıştır.

3.5. Özellik Seçimi ve Boyut Azaltma

Bu çalışmada, URL’lerden türetilen yaklaşık 54 farklı lexical özellik başlangıçta model eğitime aday değişkenler olarak belirlenmiştir. Ancak yüksek boyutlu veri yapıları, özellikle bazı özelliklerin birbirleriyle yüksek derecede ilişkili olması durumunda, model performansını olumsuz etkileyebilmektedir. Bu tür durumlar, modelin gereksiz bilgi öğrenmesine, aşırı öğrenme (overfitting) problemlerine ve hesaplama maliyetlerinin artmasına neden olmaktadır. Bu nedenle, modelin daha etkin ve genellenebilir sonuçlar üretebilmesi için özellik seçimi ve boyut azaltma süreci kritik bir adım olarak ele alınmıştır.

Özellik seçimi süreci iki aşamalı olarak gerçekleştirilmiştir. İlk aşamada, veri setindeki özellikler arasındaki doğrusal ilişkileri incelemek amacıyla korelasyon analizi uygulanmıştır. Bu analiz kapsamında, özellikler arasındaki korelasyon katsayıları hesaplanmış ve belirli bir eşik değerin üzerinde ilişkiye sahip olan özellikler tespit edilmiştir. Yüksek korelasyona sahip özellikler, veri setinde redundant (tekrarlı) bilgi taşıdığı için bu özelliklerden yalnızca biri korunmuş, diğerleri veri setinden çıkarılmıştır. Bu işlem, modelin gereksiz ve tekrar eden bilgileri öğrenmesini engelleyerek daha sade ve anlamlı bir veri yapısı oluşturulmasını sağlamıştır. Özelliklerin birbiriyle olan korelasyon değerleri Şekil 3.1’de verilmiştir. Bu hesaplama doğrultusunda ilk özellik azaltımı gerçekleştirilmiştir.



Şekil 3.1. Korelasyon sonuçları

İkinci aşamada ise, özelliklerin hedef değişken (phishing/ legitimate) ile olan ilişkisini ölçmek amacıyla Mutual Information (Karşılıklı Bilgi) yöntemi kullanılmıştır. Mutual Information, bir özelliğin hedef değişken hakkında ne kadar bilgi taşıdığını ölçen güçlü bir istatistiksel yöntemdir ve doğrusal olmayan ilişkileri de yakalayabilmesi açısından önemli bir avantaj sunmaktadır. Bu kapsamda, her bir özelliğin hedef değişken ile olan bilgi katkısı hesaplanmış ve düşük bilgi taşıyan özellikler veri setinden elenmiştir.

Korelasyon analizi ve Mutual Information yöntemlerinin birlikte kullanılması hem özellikler arası bağımlılıkların azaltılmasını hem de hedef değişken ile güçlü ilişkiye sahip özelliklerin korunmasını sağlamıştır. Bu iki aşamalı yaklaşım sayesinde, başlangıçta geniş ve karmaşık olan özellik kümesi daha kompakt, ayırt edici ve model performansını artıracak bir alt kümeye indirgenmiştir.

Uygulanan özellik seçimi süreci sonucunda, modelin eğitim süresi kısaltılmış, hesaplama maliyetleri azaltılmış ve gereksiz özelliklerin neden olabileceği gürültü ortadan kaldırılmıştır. Ayrıca, elde edilen daha sade özellik yapısı sayesinde modelin genelleme kabiliyeti artırılmış ve aşırı öğrenme riski minimize edilmiştir.

Bu iki adımlı özellik azaltımı sonucunda modelleri eğitmek için kullanılan özellikler, gruplanmış ve genel olarak anlatılmış halde Tablo 3.1’de paylaşılmıştır.

Tablo 3.1. Özellik kategorileri ve tanımı

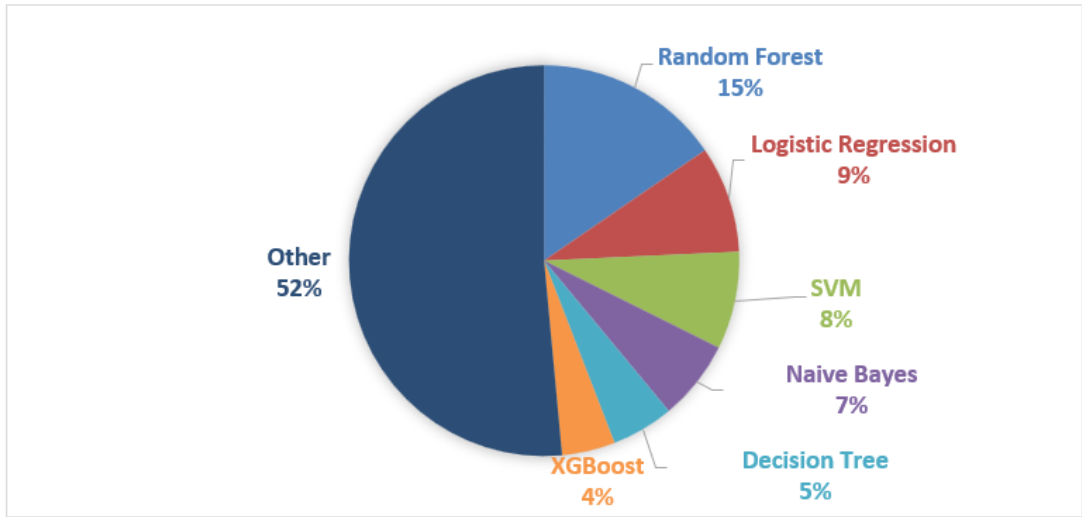
Kategori	Özellikler	Tanım
Uzunluğa dayalı	Domain_Length_Of_URL, Path_Length, fd_length, Tld_Length, Query_Length, Longest_Word, Longest_Word_in_Hostname	Bu, URL'nin alt bileşenlerinin (alan adı, yol, üst düzey alan adı, sorgu ve kelime uzunlukları) karakter tabanlı uzunluklarını ve URL'nin toplam uzunluğunu ifade eder.
Sayıma dayalı	Count_www, Count_/, Count_Digit, Count_Letter, Count_Dot, Count_-, Count_?, Count_&, Count_\t, Base64_Pattern_Cnt, Count_Embed_Domain	Bu, URL içinde yer alan özel, sayısal ve alfabetik karakterlerin sıklığını temsil eder. Kimlik avı URL'leri genellikle bu karakterleri yoğun ve manipülatif bir şekilde kullanır.
Oran bazlı	Domain_URL_Ratio, URL/Path, Digit/Letter, Vowel/Consonant, Uppercase_Lowercase_Ratio, Special_Char_Alphabet_Ratio	Bu, URL içindeki farklı karakter türlerinin birbirlerine oranlarını ifade eder. Bu oranlar, URL'nin doğal yapısından sapmaları belirlemede önemli bir rol oynar.
Etki alanı yapısı bazlı	Subdomain, Having_Path, Host_Precense_Of_Digit, use_of_ip_address, Port_Number	Bunlar, URL'nin alan adı yapısını, alt alan adlarının kullanımını, IP adresi içerip içermediğini ve port bilgilerini analiz eden yapısal özelliklerdir.
Karmaşık ölümüne dayalı	ShannonEntropy, Kolmogorov_Complexity, FractalDimension, Character_Repetition, Average_Word	Bunlar, URL dizisinin düzensizliğini, rastgeleliğini ve karakter tekrar kalıplarını ölçen istatistiksel özelliklerdir. Bu ölçümler, anormal ve yapay URL'leri tespit etmede etkilidir.
Etiket	Label	Bu, URL'nin sınıfını (zararlı veya zararsız) temsil eden bağımlı değişkendir.

Sonuç olarak, bu çalışmada uygulanan özellik seçimi ve boyut azaltma yaklaşımı, phishing URL tespitinde kullanılan makine öğrenmesi modellerinin performansını artıran ve sistemin daha verimli çalışmasını sağlayan önemli bir bileşen olarak öne çıkmaktadır. İlerleyen bölümlerde özellik azaltımı olmadan eğitilen modellerle, özellik azaltımı yapılarak eğitilen modellerin performanslarının paylaşıldığı tablolar paylaşılmıştır.

4. MODEL GELİŞTİRME VE GERÇEK ZAMANLI SİSTEM TASARIMI

4.1. Kullanılan Makine Öğrenmesi Algoritmaları

Bu bölümde, önerilen sistemin performansını değerlendirmek ve URL tabanlı kimlik avı (phishing) saldırılarını yüksek doğruluk ve düşük gecikme süresi ile tespit edebilmek amacıyla farklı makine öğrenmesi algoritmaları kullanılmıştır. Algoritma seçimi sürecinde literatürde phishing tespiti alanında yaygın olarak kullanılan, farklı öğrenme paradigmalarını temsil eden ve hem doğruluk hem de hesaplama maliyeti açısından dengeli sonuçlar sunan yöntemler tercih edilmiştir. Literatürde kullanılan algoritmaların dağılımının grafiği Şekil 4.1’de verilmiştir [23-69]. Bu yaklaşım sayesinde hem doğrusal modellerin hem de doğrusal olmayan ve ansambl (ensemble) yöntemlerin karşılaştırmalı analizi yapılabilmektedir. Özellikle gerçek zamanlı sistem tasarımı hedeflendiği için yalnızca sınıflandırma başarımı değil, aynı zamanda modelin çıkarım süresi (inference time), hesaplama karmaşıklığı ve sistem entegrasyonuna uygunluğu da dikkate alınmıştır.



Şekil 4.1. Algoritma kullanım oranı

Algoritma seçiminde üç temel kriter ön planda tutulmuştur: (i) modelin sınıflandırma performansı (accuracy, precision, recall, F1-score gibi metrikler), (ii) gerçek zamanlı uygulamalarda kritik olan hesaplama maliyeti ve hızlı tahmin üretebilme kapasitesi, (iii) yüksek boyutlu, heterojen ve çoğu zaman doğrusal olmayan ilişkiler içeren URL

tabanlı özellikler üzerinde genelleme yeteneği. Bununla birlikte phishing veri setlerinin çoğunlukla dengesiz (imbalanced) yapıda olabilmesi nedeniyle, modeller yalnızca genel doğruluk oranına göre değil, aynı zamanda yanlış pozitif (false positive) ve yanlış negatif (false negative) oranları açısından da değerlendirilmiştir. Bu kapsamda, farklı model ailelerini temsil eden toplam sekiz algoritma çalışmaya dahil edilmiştir.

Lojistik Regresyon (Logistic Regression), ikili sınıflandırma problemlerinde temel bir referans modeli olarak kullanılmıştır. Bu model, bağımsız değişkenler ile hedef değişken arasındaki ilişkiyi sigmoid fonksiyonu aracılığıyla olasılıksal bir çerçevede ifade eder. URL özelliklerinin bazıları ile hedef değişken arasında doğrusal ilişkiler bulunabileceği varsayımıyla, modelin hem hızlı çalışması hem de yüksek yorumlanabilirlik sunması önemli bir avantaj sağlamıştır. Ayrıca düşük hesaplama maliyeti sayesinde gerçek zamanlı sistemler için güçlü bir başlangıç noktası oluşturmuştur.

Destek Vektör Makineleri (Support Vector Machines - SVM), özellikle yüksek boyutlu veri setlerinde etkili performans sergileyen güçlü bir sınıflandırma algoritmasıdır. Temel amacı, sınıflar arasındaki marjini maksimize eden optimal hiper düzlemi bulmaktır. Kernel fonksiyonları sayesinde doğrusal olmayan veri yapılarında da etkili sonuçlar üretebilmesi, URL tabanlı özellikler arasındaki karmaşık ilişkilerin modellenmesinde önemli bir avantaj sağlamıştır. Ancak parametre ayarlarının hassas olması ve büyük veri setlerinde eğitim süresinin artabilmesi dikkate alınması gereken unsurlar arasındadır.

K-En Yakın Komşu (K-Nearest Neighbors-KNN), parametrik olmayan ve örnek tabanlı bir öğrenme algoritmasıdır. Bu yöntemde model, eğitim aşamasında açık bir öğrenme gerçekleştirmez bunun yerine, tahmin aşamasında yeni gelen örneğin en yakın komşularına bakarak sınıflandırma yapar. Veri dağılımını doğrudan yansıtmaları açısından avantajlı olmakla birlikte, özellikle büyük veri setlerinde hesaplama maliyetinin artması ve gerçek zamanlı kullanımda gecikmeye neden olabilmesi dezavantaj olarak değerlendirilmektedir. Buna rağmen, veri yapısına duyarlı bir referans yöntem olarak çalışmaya dahil edilmiştir.

Naive Bayes (NB), Bayes teoremine dayanan ve özellikler arasında koşullu bağımsızlık varsayımı yapan olasılıksal bir sınıflandırıcıdır. Bu varsayım gerçek

dünya verilerinde çoğu zaman tam olarak sağlanmasada, modelin basitliği ve hızlı tahmin üretme kapasitesi nedeniyle birçok problemde başarılı sonuçlar verdiği bilinmektedir. URL tabanlı phishing tespitinde, özellikle düşük hesaplama maliyeti ve hızlı çıkarım süresi avantajları nedeniyle değerlendirilmiştir.

Karar Ağaçları (Decision Trees), veri setini belirli karar kuralları doğrultusunda dallandırarak sınıflandırma yapan sezgisel ve yorumlanabilir bir yöntemdir. Model, karar verme sürecini ağaç yapısı üzerinden temsil ettiği için kullanıcıya açık ve anlaşılır bir yapı sunar. Doğrusal olmayan ilişkileri modelleyebilme yeteneği ve veri ön işleme gereksiniminin düşük olması, bu algoritmanın tercih edilmesinde önemli rol oynamıştır. Ancak aşırı öğrenmeye yatkın olması, tek başına kullanımında dikkat edilmesi gereken bir durumdur.

Rastgele Orman (Random Forest), birden fazla karar ağacının bir araya getirilmesiyle oluşturulan güçlü bir ansambl yöntemdir. Bagging yaklaşımı sayesinde her bir ağaç farklı veri alt kümeleri üzerinde eğitilir ve sonuçlar birleştirilerek daha kararlı tahminler elde edilir. Bu yapı, modelin aşırı öğrenme problemini azaltmasına ve genelleme kabiliyetini artırmasına olanak tanır. URL tabanlı phishing tespitinde yüksek doğruluk oranı ve dayanıklılığı nedeniyle önemli bir aday model olarak değerlendirilmiştir.

Gradient Boosting Machines (GBM), zayıf öğrencilerin ardışık olarak eğitildiği ve her yeni modelin bir öncekinin hatalarını minimize etmeye çalıştığı boosting tabanlı bir yöntemdir. Bu yaklaşım, özellikle karmaşık veri yapılarında yüksek performans elde edilmesini sağlar. Ancak eğitim sürecinin diğer yöntemlere göre daha uzun sürmesi ve hiperparametre ayarlarının hassas olması, gerçek zamanlı sistemler açısından dikkatle ele alınması gereken faktörlerdir.

XGBoost (Extreme Gradient Boosting), gradient boosting yaklaşımının optimize edilmiş ve daha verimli bir versiyonu olarak çalışmada yer almıştır. Düzenlileştirme (regularization) mekanizmaları sayesinde aşırı öğrenmeyi kontrol altına alırken, paralel işlem desteği ile eğitim süresini önemli ölçüde azaltmaktadır. Ayrıca eksik verilerle başa çıkabilme yeteneği ve yüksek ölçeklenebilirliği sayesinde büyük veri setlerinde üstün performans göstermektedir. Bu özellikleri nedeniyle çalışmada en güçlü modellerden biri olarak öne çıkmıştır.

Sonuç olarak, bu bölümde ele alınan sekiz farklı makine öğrenmesi algoritması hem teorik çeşitlilik hem de pratik uygulanabilirlik açısından kapsamlı bir değerlendirme yapılmasına olanak sağlamıştır. Farklı model ailelerinin karşılaştırılması sayesinde, geliştirilen gerçek zamanlı sistem için en uygun algoritmanın belirlenmesi hedeflenmiş ve sistemin hem doğruluk hem de hız açısından optimize edilmesine katkı sağlanmıştır. İlerleyen bölümlerde bu algoritmaların doğruluk ve performans sonuçları detaylı bir şekilde sunulmuştur.

4.2. Model Eğitimi, Doğrulama ve Optimizasyon

Bu çalışmada geliştirilen makine öğrenmesi modellerinin eğitimi, doğrulanması ve optimize edilmesi süreci sistematik ve çok aşamalı bir yapı içerisinde gerçekleştirilmiştir. Model geliştirme sürecinde temel amaç, yalnızca yüksek doğruluk oranı elde etmek değil, aynı zamanda gerçek zamanlı sistemlerde kullanılabilecek düşük maliyetli ve hızlı çalışan modeller elde etmektir. Bu doğrultuda hem model performansı hem de hesaplama maliyetleri birlikte değerlendirilmiştir.

Model eğitimi sürecinde, oluşturulan veri seti eğitim ve test olmak üzere iki ayrı alt kümeye ayrılmıştır. Bu ayrım sayesinde modellerin yalnızca eğitim verisi üzerindeki başarımları değil, daha önce görmediği veriler üzerindeki genelleme yeteneği de ölçülmüştür. Eğitim sürecinde kullanılan özellikler, önceki bölümde detaylandırılan iki aşamalı özellik seçimi (Correlation + Mutual Information) sonucunda elde edilen optimize edilmiş özellik kümesi ile ayrıca özellik azaltımı uygulanmamış ham özellik kümesi olmak üzere iki farklı senaryo kapsamında değerlendirilmiştir. Bu yaklaşım sayesinde özellik seçiminin model performansı ve sistem maliyetleri üzerindeki etkisi karşılaştırmalı olarak analiz edilmiştir.

Model doğrulama sürecinde, yalnızca tek bir eğitim-test bölünmesi ile elde edilen sonuçlara bağlı kalınmamış, aynı zamanda çapraz doğrulama (cross-validation) yöntemi kullanılarak modellerin genelleme performansı daha güvenilir bir şekilde ölçülmüştür. Bu kapsamda elde edilen çapraz doğrulama sonuçları (CV Accuracy, CV F1-score ve CV ROC-AUC) modellerin farklı veri alt kümeleri üzerindeki kararlılığını değerlendirmek amacıyla kullanılmıştır. Bu yaklaşım, özellikle aşırı öğrenme (overfitting) riskinin tespit edilmesi açısından kritik öneme sahiptir.

Model performansının değerlendirilmesinde yalnızca doğruluk (accuracy) metriği ile sınırlı kalınmamış, çok boyutlu bir değerlendirme yaklaşımı benimsenmiştir. Bu

kapsamda precision, recall, F1-score ve ROC-AUC gibi sınıflandırma başarımını detaylı şekilde ölçen metrikler kullanılmıştır. Özellikle phishing tespiti gibi güvenlik odaklı problemlerde yanlış negatif (false negative) sonuçların kritik riskler oluşturması nedeniyle recall ve F1-score metrikleri ayrıca önemsenmiştir. Bununla birlikte modellerin gerçek zamanlı sistemlerde kullanılabilirliğini değerlendirmek amacıyla eğitim süresi (training time), tahmin süresi (prediction time), bellek kullanımı (memory usage) ve işlemci kullanımı (CPU usage) gibi performans ve maliyet odaklı metrikler de analiz edilmiştir.

Model optimizasyonu sürecinde, her algoritma için temel hiperparametre ayarları literatürde önerilen değerler ve deneysel gözlemler doğrultusunda belirlenmiştir. Özellikle ağaç tabanlı ve boosting algoritmalarında model karmaşıklığını kontrol eden parametreler dikkate alınarak aşırı öğrenmenin önüne geçilmeye çalışılmıştır. Bununla birlikte, özellik seçimi süreci de dolaylı bir optimizasyon yöntemi olarak değerlendirilmiş ve gereksiz veya düşük katkı sağlayan özelliklerin modelden çıkarılmasıyla hem performans hem de hesaplama maliyetleri iyileştirilmiştir.

Bu kapsamda yürütülen eğitim, doğrulama ve optimizasyon süreci sonucunda, farklı makine öğrenmesi algoritmalarının hem doğruluk hem de sistem kaynak kullanımı açısından karşılaştırılmasına olanak sağlayan bütüncül bir değerlendirme altyapısı oluşturulmuştur. Elde edilen deneysel sonuçlar ve modeller arası performans karşılaştırmaları bir sonraki bölümde detaylı olarak sunulmaktadır.

4.3. Gerçek Zamanlı Sistem Mimarisi

Bu çalışmada geliştirilen sistem, URL tabanlı kimlik avı tespitini otomatik ve gerçek zamanlı olarak gerçekleştirebilen, istemci-sunucu (client-server) mimarisi üzerine kurulmuş hibrit bir yapıdan oluşmaktadır. Sistem, kullanıcı herhangi bir web sitesine eriştiği anda devreye girerek URL'yi analiz etmekte ve milisaniyeler seviyesinde sonuç üreterek kullanıcıyı bilgilendirmeyi amaçlamaktadır. Bu doğrultuda mimari tasarım sürecinde düşük gecikme süresi (low latency), yüksek doğruluk, ölçeklenebilirlik ve modülerlik temel kriterler olarak ele alınmıştır.

Önerilen sistem, temel olarak iki ana bileşenden oluşmaktadır: istemci tarafında çalışan tarayıcı eklentisi (client-side extension) ve makine öğrenmesi modelinin çalıştığı sunucu tarafı (backend inference server). Bu iki bileşen, HTTP tabanlı API

iletişimi üzerinden birbirine bağlıdır ve birlikte uçtan uca bir gerçek zamanlı analiz hattı oluşturmaktadır.

Sistemin istemci tarafını oluşturan tarayıcı eklentisi, kullanıcının ziyaret ettiği web sayfalarını sürekli olarak izleyen ve URL değişimlerini otomatik olarak tespit eden bir yapıya sahiptir. Bu yapı, tarayıcı olaylarını (tab update, navigation event vb.) dinleyerek kullanıcı herhangi bir sayfaya giriş yaptığında ilgili URL bilgisini anlık olarak elde etmektedir. Böylece sistem, kullanıcı müdahalesine ihtiyaç duymadan pasif bir güvenlik katmanı olarak çalışmaktadır.

Elde edilen URL, istemci tarafında herhangi bir ağır işlem yapılmadan doğrudan sunucuya iletilmektedir. Bu tasarım kararı, tarayıcı tarafında hesaplama yükünü minimize etmek ve sistemin hızlı tepki vermesini sağlamak amacıyla alınmıştır. Ayrıca bu yaklaşım, modelin güncellenmesi veya değiştirilmesi durumunda istemci tarafında herhangi bir değişiklik yapılmasına gerek bırakmamaktadır.

Sunucu tarafı, sistemin çekirdek analiz bileşenini oluşturmaktadır. Bu katmanda, Python tabanlı bir servis mimarisi kullanılarak URL işleme ve sınıflandırma işlemleri gerçekleştirilmektedir. Sunucuya iletilen URL ilk olarak eğitim verisindeki format tutarlılığını sağlamak amacıyla normalize edilmektedir. Bu adımda subdomain öneki (www.) ve gereksiz path karakterleri standart hale getirilmektedir. Bu işlemten sonra özellik çıkarım (feature extraction) sürecine geçilmektedir. Bu aşamada URL, domain, subdomain, path ve query gibi bileşenlerine ayrılarak uzunluk, karakter dağılımı, oranlar ve karmaşıklık metrikleri gibi çok boyutlu özellikler elde edilir. Bu özellikler, modelin giriş vektörünü oluşturacak şekilde yapılandırılır.

Oluşturulan özellik vektörü, daha önce eğitilmiş ve optimize edilmiş makine öğrenmesi modeline (bu çalışma kapsamında XGBoost) iletilir. Model, giriş verisini işleyerek URL'nin phishing veya legitimate sınıfına ait olma olasılığını hesaplar ve sınıflandırma sonucunu üretir. Bu işlem yalnızca çıkarım (inference) aşamasını kapsadığı için oldukça hızlı bir şekilde gerçekleştirilmektedir. Elde edilen sonuç, JSON formatında istemciye geri gönderilir.

İstemci tarafında alınan bu sonuç, kullanıcıya görsel bir geri bildirim olarak sunulmaktadır. Bu geri bildirim, tarayıcı üzerinde uyarı mesajı şeklinde sayfa üzerinde görsel uyarı banner'ı ve eklenti arayüzü ile gösterilmektedir. Bu sayede kullanıcı,

ziyaret ettiđi web sayfasının potansiyel risk durumu hakkında anında bilgilendirilir ve gerekli önlemleri alabilir.

Önerilen mimarinin en önemli avantajlarından biri, iş yükünün istemci ve sunucu arasında dengeli bir şekilde dağıtılmış olmasıdır. Hesaplama açısından maliyetli işlemler (özellik çıkarımı ve model tahmini) sunucu tarafında gerçekleştirilirken, istemci tarafı yalnızca veri iletimi ve kullanıcı etkileşimi ile sınırlı tutulmuştur. Bu durum, sistemin farklı cihazlarda (düşük donanımlı bilgisayarlar dahil) stabil bir şekilde çalışmasını sağlamaktadır.

Ayrıca sistem, modüler bir yapıya sahip olacak şekilde tasarlanmıştır. Bu sayede model güncellemeleri, yeni özelliklerin eklenmesi veya farklı algoritmaların sisteme entegrasyonu yalnızca sunucu tarafında yapılabilmektedir. Bu yaklaşım, sistemin sürdürülebilirliğini ve bakım kolaylığını önemli ölçüde artırmaktadır.

Gerçek zamanlılık açısından değerlendirildiğinde, sistemin performansı büyük ölçüde ağ gecikmesi ve model çıkarım süresine bağlıdır. Ancak URL tabanlı analiz yaklaşımı sayesinde veri boyutunun küçük olması ve modelin optimize edilmiş yapısı, toplam işlem süresinin oldukça düşük seviyelerde tutulmasını sağlamaktadır. Bu da sistemin pratik kullanım senaryolarında etkin bir şekilde kullanılabileceğini göstermektedir.

Sonuç olarak geliştirilen bu istemci-sunucu tabanlı gerçek zamanlı sistem mimarisi, makine öğrenmesi modellerinin tarayıcı ortamında etkin ve ölçeklenebilir bir şekilde kullanılabileceğini göstermektedir. Sistem hem akademik hem de pratik açıdan uygulanabilir bir çözüm sunmakta olup, phishing tespitine yönelik modern ve etkili bir yaklaşım ortaya koymaktadır.

4.4. Tarayıcı Eklentisi Geliştirme Süreci

Bu çalışma kapsamında geliştirilen makine öğrenmesi tabanlı phishing tespit sisteminin kullanıcıya doğrudan ve pratik bir şekilde sunulabilmesi amacıyla, Google Chrome tabanlı bir tarayıcı eklentisi geliştirilmiştir. Geliştirilen eklenti, kullanıcı herhangi bir web sitesine eriştiđi anda otomatik olarak devreye giren ve URL'yi arka planda analiz sürecine dahil eden bir yapı üzerine kurulmuştur. Bu yaklaşım, kullanıcı müdahalesine ihtiyaç duymadan çalışan pasif bir güvenlik mekanizması sağlamaktadır.

Eklenti geliştirme sürecinde, modern tarayıcı mimarileri ile uyumlu olması nedeniyle Chrome Extension altyapısı kullanılmış ve eklenti, güncel standartlara uygun olarak Manifest V3 yapısı ile yapılandırılmıştır. Sistem, minimum izin prensibi doğrultusunda yalnızca gerekli yetkiler ile çalışacak şekilde tasarlanmıştır. Bu sayede hem güvenlik hem de kullanıcı gizliliği açısından kontrollü bir yapı elde edilmiştir.

Sistemde URL yakalama işlemi, tarayıcı sekmelerinde gerçekleşen sayfa yüklenme ve yönlendirme olaylarını izleyen bir mekanizma üzerinden gerçekleştirilmektedir. Kullanıcı yeni bir web sayfasına giriş yaptığında, ilgili URL bilgisi otomatik olarak elde edilmekte ve analiz süreci başlatılmaktadır. Bu yapı sayesinde sistem, sürekli çalışan ve kullanıcıdan bağımsız olarak işlem yapan gerçek zamanlı bir güvenlik katmanı haline gelmektedir.

Elde edilen URL bilgisi, istemci tarafında herhangi bir karmaşık işlem uygulanmadan doğrudan arka planda çalışan sunucuya iletilmektedir. Bu iletişim, HTTP tabanlı istekler aracılığıyla gerçekleştirilmekte olup, veri JSON formatında taşınmaktadır. Gönderilen veri yalnızca URL bilgisini içermektedir. Bu yaklaşım, veri boyutunun düşük tutulmasını sağlarken aynı zamanda sistemin hızlı çalışmasına katkı sunmaktadır.

Sunucu tarafında gerçekleştirilen analiz süreci kapsamında, gelen URL üzerinde Python dili kullanılarak geliştirilen özel betikler aracılığıyla özellik çıkarımı (feature extraction) işlemleri uygulanmaktadır. Bu aşamada URL, domain, subdomain, path ve query gibi bileşenlerine ayrılmakta uzunluk tabanlı, karakter frekansına dayalı, oran tabanlı ve karmaşıklık ölçümlerine dayanan özellikler doğrudan yazılan Python betikleri ile hesaplanmaktadır. Elde edilen değerler, modelin giriş formatına uygun bir özellik vektörüne dönüştürülmektedir. Bu özellikler çıkarılırken hazır kod kullanılmamış olup özellik hesaplama mantığı standart Python kütüphaneleri (urllib, re, numpy vb.) kullanılarak bu çalışma kapsamında özgün biçimde geliştirilmiştir.

Oluşturulan özellik vektörü, daha önce eğitilmiş makine öğrenmesi modeline aktarılmaktadır. Model tarafından üretilen çıktı, URL'nin zararlı olma olasılığını ifade eden bir skor değeri ile birlikte doğrudan sınıflandırma kararını (phishing / legitimate) içermektedir. Nihai karar, modelin kendi iç karar mekanizması tarafından belirlenmektedir.

Analiz sonucunun istemciye iletilmesinin ardından, yalnızca zararlı olarak sınıflandırılan URL'ler için kullanıcıya geri bildirim sağlanmaktadır. Bu geri bildirim, tarayıcı üzerinde görüntülenen bir banner aracılığıyla sunulmakta ve kullanıcı potansiyel bir phishing saldırısına karşı bilgilendirilmektedir. Bu banner Şekil 4.2'de gösterilmektedir. Güvenli olarak değerlendirilen URL'ler için herhangi bir bildirim yapılmaması, kullanıcı deneyimini olumsuz etkileyebilecek gereksiz uyarıların önüne geçmek amacıyla tercih edilmiştir.



Şekil 4.2. Banner

Geliştirilen eklenti mimarisi, istemci ve sunucu bileşenlerinin ayrılması sayesinde modüler bir yapı sunmaktadır. Bu sayede makine öğrenmesi modeli, özellik çıkarım yöntemleri veya karar mekanizması üzerinde yapılacak güncellemeler yalnızca sunucu tarafında gerçekleştirilerek sistemin bakım ve güncellenebilirliğini kolaylaştırılmıştır. Aynı zamanda istemci tarafının hafif tutulması, sistemin farklı donanım seviyelerine sahip cihazlarda sorunsuz çalışabilmesini sağlamaktadır.

Sonuç olarak, geliştirilen tarayıcı eklentisi, makine öğrenmesi tabanlı phishing tespit sisteminin gerçek zamanlı ve kullanıcı dostu bir şekilde uygulanmasını mümkün kılmakta ve teorik modelin pratik bir güvenlik çözümüne dönüştürülmesini sağlamaktadır.

4.5. Güvenlik ve Performans Değerlendirme

Bu bölümde, geliştirilen gerçek zamanlı phishing tespit sisteminin güvenlik ve performans açısından değerlendirilmesi yapılmaktadır. Sistem yalnızca sınıflandırma başarımı açısından değil, aynı zamanda gerçek zamanlı çalışabilirlik, kaynak kullanımı, ölçeklenebilirlik ve kullanıcı gizliliği gibi kritik kriterler çerçevesinde ele alınmıştır.

Performans açısından değerlendirildiğinde, sistemin çalışma süreci üç temel aşamadan oluşmaktadır: URL'nin tarayıcı tarafından yakalanması, sunucuya iletilmesi ve model tarafından analiz edilmesi. URL tabanlı yaklaşım sayesinde gönderilen veri boyutunun oldukça küçük olması, ağ üzerinden veri iletim süresini minimum seviyede tutmaktadır. Sunucu tarafında gerçekleştirilen özellik çıkarımı ve model çıkarım

işlemleri, yalnızca URL metni üzerinde çalıştığı için yüksek hesaplama maliyeti gerektirmemekte ve hızlı bir şekilde tamamlanabilmektedir. Bu durum, sistemin gerçek zamanlıya yakın bir performans sergilemesini sağlamaktadır.

Sistemde hesaplama açısından en maliyetli işlemler olan özellik çıkarımı ve model tahmini sunucu tarafında gerçekleştirilmektedir. Bu tasarım tercihi sayesinde istemci tarafındaki yük minimum seviyede tutulmuş ve sistemin farklı donanım seviyelerine sahip cihazlarda sorunsuz çalışabilmesi sağlanmıştır. Ayrıca merkezi bir sunucu üzerinde çalışılması, model güncellemelerinin ve iyileştirmelerin istemci tarafına müdahale edilmeden yapılabilmesine olanak tanımaktadır.

Sistemin bu performans hedeflerini ne kadar karşıladığını belirlemek için 177 farklı URL içeren bir test veri seti üzerinde yapılan saha testi sonuçları, sistemin ortalama 16,517 milisaniye gibi oldukça düşük bir gecikme ile çalıştığını ve %99,44 doğruluk oranına ulaştığını kanıtlamıştır. Bu deneysel bulgulara ve eklentinin gerçek zamanlı çalışma performansına dair detaylı analizlere Tablo 5.3'te yer verilmiştir.

Sistem performansını etkileyen önemli faktörlerden bir diğeri ağ gecikmesidir. Ancak iletilen verinin yalnızca URL ile sınırlı olması ve veri boyutunun düşük olması sayesinde bu gecikmenin sınırlı kaldığı değerlendirilmektedir. Ayrıca modelin optimize edilmiş yapısı, çıkarım süresinin kısa olmasını sağlayarak toplam sistem gecikmesini azaltmaktadır. Bu özellikler, sistemin gerçek kullanım senaryolarında kullanıcı deneyimini olumsuz etkilemeden çalışabilmesini mümkün kılmaktadır.

Güvenlik açısından değerlendirildiğinde, sistemin yalnızca URL bilgisi üzerinden çalışması önemli bir avantaj sağlamaktadır. Web sayfası içeriğinin analiz edilmemesi ve herhangi bir kullanıcı verisinin toplanmaması, kullanıcı gizliliğinin korunmasına katkı sunmaktadır. Ayrıca sistemin üçüncü taraf servis veya harici güvenlik API'lerine bağımlı olmaması, veri paylaşım risklerini azaltmaktadır.

Sistemin istemci-sunucu mimarisi üzerine kurulmuş olması, veri iletimi sürecinde güvenli iletişim gereksinimini ortaya çıkarmaktadır. Bu kapsamda, istemci ile sunucu arasındaki veri alışverişinin güvenli protokoller üzerinden gerçekleştirilmesi önerilmektedir. Mevcut çalışmada temel işlevsellik ve sistem mimarisi ön planda tutulmuş olup, güvenli iletişim mekanizmaları ve ek doğrulama katmanları ilerleyen geliştirme aşamalarında ele alınabilecek konular arasında değerlendirilmektedir.

Sistemin kullanıcı deneyimi açısından değerlendirilmesinde, yalnızca zararlı olarak sınıflandırılan URL'ler için uyarı verilmesi önemli bir tasarım kararıdır. Bu yaklaşım, kullanıcıyı gereksiz bildirimlerle rahatsız etmemekte ve sistemin daha etkili bir şekilde kullanılmasını sağlamaktadır. Aynı zamanda yanlış pozitif (false positive) durumlarının kullanıcı üzerindeki olumsuz etkisini minimize etmeye yönelik bir denge sunmaktadır.

Sonuç olarak geliştirilen sistem, düşük veri iletimi, hızlı analiz süreci ve optimize edilmiş model yapısı sayesinde performans açısından gerçek zamanlı kullanım gereksinimlerini karşılayabilecek düzeydedir. Güvenlik açısından ise kullanıcı gizliliğini koruyan, dış bağımlılıkları minimize eden ve kontrollü veri akışı sağlayan bir yapı sunmaktadır. Bu özellikler, sistemin hem akademik hem de pratik uygulamalarda kullanılabilir bir çözüm olduğunu göstermektedir.

5. SONUÇLAR VE DEĞERLENDİRME

5.1. Deneysel Bulgular

Bu bölümde, geliştirilen makine öğrenmesi modellerinin performansına ilişkin deneysel sonuçlar sunulmaktadır. Modeller, iki farklı veri yapısı üzerinde eğitilerek değerlendirilmiştir. İlk senaryoda, korelasyon analizi ve Mutual Information yöntemleri kullanılarak gerçekleştirilen iki aşamalı özellik seçimi süreci sonucunda elde edilen optimize edilmiş özellik kümesi kullanılmıştır. İkinci senaryoda ise herhangi bir özellik azaltımı uygulanmadan, literatürden elde edilen ve veri setindeki sadece URL den çıkarılan özellikler kullanılarak model eğitimi gerçekleştirilmiştir.

Bu iki farklı yaklaşım sayesinde, özellik seçimi sürecinin model performansı ve sistem kaynak kullanımı üzerindeki etkisi deneysel olarak gözlemlenmiştir. Modellerin değerlendirilmesinde yalnızca sınıflandırma başarımı değil, aynı zamanda sistem performansını doğrudan etkileyen eğitim süresi, tahmin süresi, bellek kullanımı ve işlemci kullanımı gibi metrikler de dikkate alınmıştır.

Özellik azaltımı uygulanmış veri seti üzerinde elde edilen model sonuçları Tablo 5.1’de sunulmaktadır. Bu tabloda, modellerin test doğruluğu (Test Accuracy), precision, recall, F1-score, ROC-AUC değerlerinin yanı sıra çapraz doğrulama sonuçları ve sistem kaynak kullanımına ilişkin metrikler yer almaktadır.

Tablo 5.1 incelendiğinde, modellerin büyük çoğunluğunun oldukça yüksek doğruluk, kesinlik, duyarlılık ve F1 skoru değerlerine ulaştığı görülmektedir. Özellikle ensemble öğrenme tabanlı XGBoost, Random Forest ve Gradient Boosting algoritmaları test performansı bakımından en başarılı modeller arasında yer almaktadır. Bununla birlikte, modeller arasında eğitim süresi, tahmin süresi ve hesaplama kaynaklarının kullanımı açısından önemli farklılıklar bulunduğu görülmektedir. Bu durum, model seçiminde yalnızca sınıflandırma başarısının değil, hesaplama maliyetinin de dikkate alınması gerektiğini göstermektedir.

Özellik azaltımı uygulanmadan elde edilen model sonuçları ise Tablo 5.2’de sunulmaktadır. Bu tabloda, modellerin ham özellik kümesi ile eğitilmesi sonucunda elde edilen performans değerleri yer almaktadır.

Tablo 5.2’de yer alan sonuçlar incelendiğinde, modellerin doğruluk değerlerinin genel olarak yüksek seviyelerde kaldığı, ancak sistem kaynak kullanımı ve işlem süreleri açısından farklılıkların ortaya çıktığı görülmektedir. Özellikle eğitim süresi ve bellek kullanımı açısından bazı modellerde belirgin artışlar gözlemlenmektedir.

Tablo 5.1. Model eğitim sonuçları

Model	Test_Accuracy	Test_Precision	Test_Recall	Test_F1	Train_Time (s)	Predict_Time (s)	Memory_Usage (MB)	CPU_Usage (%)	Test_ROC_AUC	CV_Accuracy	CV_F1	CV_ROC_AUC
XGBoost	0,9976	0,9995	0,9958	0,9977	0,8595	0,0262	103,65	93,3	0,9988	0,9979	0,9980	0,9990
Random Forest	0,9972	0,9992	0,9954	0,9973	16,3054	0,3981	26,1	0	0,9986	0,9978	0,9979	0,9988
Logistic Regression	0,9966	0,9996	0,9939	0,9967	6,2479	0,0120	3,76	9,6	0,9985	0,9971	0,9972	0,9987
Decision Tree	0,9964	0,9976	0,9956	0,9966	1,4830	0,0151	0,3	12,7	0,9965	0,9971	0,9972	0,9971
Gradient Boosting	0,9970	0,9998	0,9946	0,9972	40,7351	0,0780	-3,56	1,4	0,9986	0,9976	0,9977	0,9989
KNN	0,9940	0,9979	0,9906	0,9942	0,0601	14,512	74,49	2	0,9974	0,9944	0,9946	0,9977
Linear SVM	0,9966	0,9998	0,9937	0,9967	3,4926	0,0097	3,99	0	0,9986	0,9972	0,9973	0,9988
Naive Bayes	0,9951	0,9963	0,9944	0,9953	0,1600	0,0397	-4,3	3,6	0,9976	0,9957	0,9959	0,9976

Tablo 5.2. Özellik azaltımı olmadan model eğitim sonuçları

Model	Test_Accuracy	Test_Precision	Test_Recall	Test_F1	Train_Time (s)	Predict_Time (s)	Memory_Usage (MB)	CPU_Usage (%)	Test_ROC_AUC	CV_Accuracy	CV_F1	CV_ROC_AUC
XGBoost	0,9976	0,9995	0,9958	0,9977	0,9017	0,0271	105,94	79,2	0,9988	0,9980	0,9981	0,9990
Random Forest	0,9975	0,9994	0,9959	0,9976	19,2124	0,4081	24,870	7	0,9985	0,9979	0,9980	0,9988
Logistic Regression	0,9973	0,9997	0,9952	0,9974	11,6056	0,0152	3,6600	3,9	0,9986	0,9977	0,9978	0,9988
Decision Tree	0,9966	0,9977	0,9958	0,9968	1,9579	0,02	0,7900	0,9	0,9967	0,9970	0,9972	0,9971
Gradient Boosting	0,9971	0,9998	0,9947	0,9972	53,6041	0,0877	-1,99	2,1	0,9987	0,9977	0,9978	0,9989
KNN	0,9934	0,9973	0,9899	0,9936	0,1135	16,816	107,06	0	0,9973	0,9941	0,9943	0,9977
Linear SVM	0,9965	0,9998	0,9934	0,9966	2,9675	0,0176	2,93	2,9	0,9981	0,9971	0,9972	0,9985
Naive Bayes	0,9234	0,9937	0,8587	0,9213	0,2524	0,0603	-1,87	1,9	0,9940	0,9242	0,9222	0,9943

Bu bölümde sunulan bulgular, özellik seçimi sürecinin model performansı ve sistem verimliliği üzerindeki etkisini ortaya koymakta olup, modeller arasındaki detaylı karşılaştırmalar ve analizler bir sonraki bölümde ele alınmaktadır.

5.2. Model Performans ve Karşılaştırmalar

Bu bölümde, farklı makine öğrenmesi algoritmalarının performansları hem özellik azaltımı uygulanmış hem de uygulanmamış veri setleri üzerinde elde edilen sonuçlar doğrultusunda karşılaştırmalı olarak analiz edilmektedir. Değerlendirme sürecinde yalnızca doğruluk (accuracy) metriği değil precision, recall, F1-score, ROC-AUC, çapraz doğrulama sonuçları ve sistem kaynak kullanımı gibi çok boyutlu performans kriterleri dikkate alınmıştır.

Özellik azaltımı uygulanmış veri seti üzerinde elde edilen sonuçlar incelendiğinde, XGBoost algoritmasının yaklaşık 0.9977 test doğruluğu, 0.9977 F1-score ve 0.9988 ROC-AUC değeri ile en başarılı modellerden biri olduğu görülmektedir. Modelin eğitim süresi yaklaşık 0,86 saniye, tahmin süresi 0,03 saniye ve bellek kullanımı 103.65 MB olarak ölçülmüştür. Ayrıca çapraz doğrulama doğruluğunun (CV Accuracy = 0.9979) test doğruluğuna oldukça yakın olması, modelin aşırı öğrenme (overfitting) eğiliminin düşük olduğunu ve yüksek genelleme yeteneğine sahip olduğunu göstermektedir. Performans ile hesaplama maliyeti birlikte değerlendirildiğinde XGBoost algoritmasının dengeli bir yapı sergilediği söylenebilir.

Random Forest algoritması da 0.997 test doğruluğu ve 0.997 F1-score ile oldukça başarılı sonuçlar elde etmiştir. Ancak eğitim süresinin 16,3 saniye, tahmin süresinin ise yaklaşık 0,4 saniye olması, XGBoost algoritmasına göre daha yüksek hesaplama maliyeti oluşturduğunu göstermektedir. Buna rağmen ROC-AUC değerinin 0.9986 olması, modelin sınıflandırma başarısının oldukça yüksek olduğunu ortaya koymaktadır.

Logistic Regression modeli 0.9966 doğruluk, 0.9996 precision ve 0.9967 F1-score değerleri ile başarılı performans sergilemiştir. Eğitim süresi (6,25 saniye) ve tahmin süresi (0,01 saniye) incelendiğinde, modelin özellikle tahmin aşamasında oldukça hızlı çalıştığı görülmektedir. Bu durum, doğrusal yapısından kaynaklanan düşük hesaplama maliyetini desteklemektedir.

Decision Tree algoritması 0.9964 doğruluk değeri ile diğer güçlü modellerin bir miktar gerisinde kalmasına rağmen, yaklaşık 1,5 saniyelik eğitim süresi ve yaklaşık 0,02 saniyelik tahmin süresi sayesinde hızlı çalışan modeller arasında yer almaktadır. Buna karşın tek bir karar ağacına dayanması nedeniyle ensemble yöntemlere göre daha sınırlı bir genelleme performansı göstermektedir.

Gradient Boosting algoritması 0.997 doğruluk ve 0.997 F1-score değerleri ile yüksek sınıflandırma başarısı göstermiştir. Ancak eğitim süresinin 40.7351 saniye olması, diğer birçok modele göre daha yüksek hesaplama maliyetine sahip olduğunu göstermektedir. Bu nedenle gerçek zamanlı uygulamalarda eğitim maliyeti açısından dezavantaj oluşturabilmektedir.

KNN algoritması 0,994 doğruluk oranına ulaşmasına rağmen 14,5 saniyelik tahmin süresi ile tüm modeller arasında en yüksek tahmin süresine sahip algoritma olmuştur. Bunun temel nedeni, KNN algoritmasının tahmin aşamasında eğitim verilerinin tamamı üzerinde benzerlik hesaplaması yapmasıdır. Bu nedenle büyük veri kümelerinde gerçek zamanlı kullanım açısından ölçeklenebilirliği sınırlıdır.

Linear SVM modeli 0.9966 doğruluk, 0.9998 precision ve 0.9967 F1-score değerleri ile başarılı performans göstermiştir. Özellikle 0.0097 saniyelik tahmin süresi tüm modeller arasında en düşük değerlerden biri olup, modelin hızlı tahmin yapabilme yeteneğini ortaya koymaktadır. Eğitim süresi de (3,5 saniye) oldukça düşük seviyededir.

Naive Bayes algoritması ise özellik azaltımı sonrasında dikkat çekici bir performans artışı göstermiştir. Model 0.9951 doğruluk, 0.9944 recall ve 0.9953 F1-score değerlerine ulaşarak diğer birçok model ile rekabet edebilecek seviyeye gelmiştir. Eğitim süresinin yalnızca 0,16 saniye olması ve tahmin süresinin 0,04 saniye olarak gerçekleşmesi, modelin hesaplama açısından oldukça verimli olduğunu göstermektedir.

Özellik azaltımı uygulanmamış veri seti üzerinde elde edilen sonuçlar incelendiğinde, modellerin doğruluk değerlerinin genel olarak özellik azaltımı uygulanmış senaryoya oldukça yakın olduğu görülmektedir. XGBoost algoritması bu veri setinde de 0.9976 doğruluk değeri ile benzer performans sergilemiştir. Random Forest (0.9975), Logistic Regression (0.9973) ve Gradient Boosting (0.9971) modelleri de benzer şekilde yüksek doğruluk değerleri elde etmiştir.

Bununla birlikte özellik azaltımı uygulanmadığında birçok modelin eğitim ve tahmin sürelerinde artış meydana gelmiştir. Random Forest algoritmasının eğitim süresi 19.21 saniyeden 16.30 saniyeye, Gradient Boosting algoritmasının eğitim süresi ise 53.60 saniyeden 40.73 saniyeye düşmüştür. Benzer şekilde Logistic Regression modelinin eğitim süresi 11.60 saniyeden 6.24 saniyeye, Decision Tree modelinin eğitim süresi 1.95 saniyeden 1.48 saniyeye gerilemiştir. XGBoost algoritmasında da eğitim süresi 0.90 saniyeden 0.85 saniyeye, tahmin süresi ise 0.0271 saniyeden 0.0262 saniyeye düşmüştür. KNN algoritmasının tahmin süresi de 16.816 saniyeden 14.512 saniyeye gerileyerek özellik azaltımının tahmin maliyetini de azalttığını göstermektedir.

Özellik azaltımı sonrasında doğruluk değerlerinde anlamlı bir performans kaybı gözlenmemesine rağmen eğitim ve tahmin sürelerinin genel olarak azalması, özellik seçimi sürecinin hesaplama maliyetini düşürürken sınıflandırma performansını büyük ölçüde koruduğunu göstermektedir. Özellikle Naive Bayes algoritmasının doğruluğunun 0,92 değerinden 0,99 değerine yükselmesi, özellik seçiminin bazı algoritmalar üzerinde yalnızca hesaplama maliyetini azaltmakla kalmayıp sınıflandırma başarısını da önemli ölçüde artırabileceğini ortaya koymaktadır.

Buna karşılık doğruluk metriklerinde anlamlı bir artış gözlemlenmemesi, özellik azaltımı sürecinin model performansını korurken sistem maliyetlerini önemli ölçüde düşürdüğünü ortaya koymaktadır. Özellikle XGBoost modeli, her iki senaryoda da benzer doğruluk değerleri sunmasına rağmen, özellik azaltımı uygulandığında daha düşük bellek kullanımı ve daha hızlı eğitim süresi ile daha verimli bir yapı sergilemiştir.

Sonuç olarak, yapılan karşılaştırmalar doğrultusunda, ensemble tabanlı yöntemlerin (özellikle XGBoost ve Random Forest) en yüksek doğruluk değerlerini sağladığı, ancak sistem performansı ve gerçek zamanlı uygulanabilirlik açısından XGBoost algoritmasının en dengeli ve en uygun model olduğu belirlenmiştir. Özellik seçimi sürecinin ise model doğruluğunu koruyarak hesaplama maliyetlerini önemli ölçüde azaltan kritik bir adım olduğu açıkça görülmektedir.

5.3. Bulguların Yorumu

Bu çalışma kapsamında elde edilen bulgular, URL tabanlı özellikler kullanılarak geliştirilen makine öğrenmesi modellerinin phishing saldırılarının tespitinde oldukça yüksek başarı oranlarına ulaşabildiğini göstermektedir. Özellikle elde edilen sonuçlar,

yalnızca URL yapısından çıkarılan özelliklerin kullanılmasıyla, herhangi bir web sayfası içeriği veya davranışsal analiz gerektirmeden yüksek doğrulukta sınıflandırma yapılabileceğini ortaya koymaktadır. Bu durum, çalışmanın temel varsayımını desteklemekte ve URL tabanlı analiz yaklaşımının gerçek zamanlı phishing tespit sistemleri için uygun bir yöntem olduğunu göstermektedir.

Elde edilen bulgular literatürde yer alan çalışmalarla genel olarak uyumludur. Özellikle ensemble tabanlı algoritmalar olan XGBoost, Random Forest ve Gradient Boosting modellerinin yüksek doğruluk, F1-score ve ROC-AUC değerlerine ulaşması, bu algoritmaların phishing URL'lerinin karmaşık örüntülerini başarılı şekilde öğrenebildiğini göstermektedir. Özellikle XGBoost algoritması, hem özellik azaltımı uygulanmış hem de uygulanmamış veri setlerinde yaklaşık %99,8 doğruluk oranını koruyarak oldukça kararlı bir performans sergilemiştir.

Çalışmanın en önemli bulgularından biri, uygulanan iki aşamalı özellik seçimi sürecinin model performansı üzerindeki etkisidir. Korelasyon analizi ve Mutual Information yöntemleri kullanılarak gerçekleştirilen özellik azaltımı sonrasında, modellerin doğruluk, precision, recall, F1-score ve ROC-AUC değerlerinde belirgin bir performans kaybı gözlenmemiştir. Buna karşın eğitim süreleri ve tahmin sürelerinde genel olarak azalma meydana gelmiştir. Özellikle Random Forest, Gradient Boosting, Logistic Regression, Decision Tree, XGBoost ve KNN modellerinde eğitim ve tahmin sürelerinin kısalması, gereksiz ve tekrarlı özelliklerin model üzerinde ek hesaplama yükü oluşturduğunu göstermektedir. Bu sonuç, uygun bir özellik seçimi sürecinin yalnızca hesaplama maliyetini azaltmakla kalmayıp model verimliliğini de artırabileceğini ortaya koymaktadır.

Dikkat çekici bulgulardan biri de Naive Bayes algoritmasının özellik seçimi sonrasında gösterdiği performans artışıdır. Özellik azaltımı uygulanmadan önce %92.34 doğruluk oranına sahip olan model, özellik seçimi sonrasında %99,51 doğruluk seviyesine ulaşmıştır. Benzer şekilde recall ve F1-score değerlerinde de önemli artışlar gözlenmiştir. Bu durum, Naive Bayes algoritmasının yüksek korelasyona sahip veya gereksiz özelliklerden diğer modellere göre daha fazla etkilendiğini ve uygun özellik seçiminin bu model üzerindeki olumlu etkisini açıkça ortaya koymaktadır.

Gerçek zamanlı sistemler açısından değerlendirildiğinde, yalnızca doğruluk oranının yeterli olmadığı, aynı zamanda modelin hızlı tahmin yapabilmesi ve makul hesaplama maliyetine sahip olması gerektiği görülmektedir. Bu açıdan XGBoost algoritması, yüksek doğruluk oranını oldukça kısa eğitim ve tahmin süreleri ile birleştirerek performans ve hesaplama maliyeti arasında en dengeli modeli oluşturmuştur. Buna karşılık Random Forest ve Gradient Boosting algoritmaları benzer doğruluk değerleri üretmelerine rağmen daha uzun eğitim sürelerine ihtiyaç duymaktadır.

KNN algoritması ise yüksek doğruluk değerine sahip olmasına rağmen tahmin süresinin diğer modellere göre oldukça yüksek olması nedeniyle gerçek zamanlı uygulamalar açısından dezavantajlıdır. Bunun temel nedeni, KNN algoritmasının tahmin aşamasında eğitim örneklerinin tamamı üzerinde benzerlik hesaplaması gerçekleştirmesidir. Bu durum, veri seti büyüdükçe tahmin süresinin önemli ölçüde artmasına neden olmakta ve modelin ölçeklenebilirliğini sınırlandırmaktadır.

Linear SVM ve Logistic Regression modelleri de yüksek doğruluk oranları elde etmiş olup özellikle kısa tahmin süreleri sayesinde gerçek zamanlı uygulamalar açısından dikkat çekmektedir. Bununla birlikte, doğruluk açısından ensemble tabanlı yöntemlerin gerisinde kalmaları, doğrusal karar sınırlarının karmaşık phishing URL örüntülerini modellemede belirli sınırlılıklara sahip olabileceğini düşündürmektedir.

Çalışmanın bir diğer önemli sonucu, geliştirilen sistem mimarisinin pratik uygulanabilirliğidir. URL tabanlı analiz yaklaşımı sayesinde sistemin web sayfası içeriğini indirmesine veya harici analiz servislerine ihtiyaç duymadan çalışabilmesi, hem kullanıcı gizliliğinin korunmasına hem de daha hızlı karar verilmesine katkı sağlamaktadır. Bu özellik, önerilen yöntemin gerçek zamanlı tarayıcı eklentileri ve ağ güvenliği uygulamaları gibi pratik kullanım alanlarında uygulanabilir olduğunu göstermektedir.

Bununla birlikte çalışmanın bazı sınırlılıkları bulunmaktadır. Sistem yalnızca URL tabanlı özellikleri kullanmaktadır. Bu nedenle URL yapısında belirgin bir anormallik bulunmayan, ancak içerik veya davranış analizi gerektiren gelişmiş phishing saldırılarının tespitinde performans sınırlamaları oluşabilir. Ayrıca modeller belirli bir veri kümesi üzerinde eğitildiğinden, farklı veri kümelerinde benzer başarının sürdürülebilmesi için yeniden eğitim veya model güncellemesi gerekmektedir.

Sonuç olarak elde edilen bulgular, URL tabanlı makine öğrenmesi yaklaşımlarının phishing saldırılarının tespitinde yüksek doğruluk, güçlü genelleme yeteneği ve gerçek zamanlı uygulanabilirlik sunduğunu göstermektedir. Özellikle özellik seçimi sürecinin hesaplama maliyetini azaltırken model performansını büyük ölçüde koruması ve bazı modellerde önemli performans artışları sağlaması, bu yaklaşımın etkinliğini ortaya koymaktadır. Genel değerlendirme sonucunda XGBoost algoritması performans, kararlılık ve hesaplama maliyeti açısından en dengeli model olarak öne çıkarken, önerilen sistemin hem akademik literatüre hem de gerçek dünya uygulamalarına katkı sağlayabilecek nitelikte olduğu değerlendirilmektedir.

5.4. Tarayıcı Eklentisi Saha Testi ve Gerçek Zamanlı Performans Analizi

Makine öğrenmesi modellerinin veri seti üzerindeki başarısı Bölüm 5.1 ve 5.2’de detaylı anlatılmıştır. Ancak sistemin gerçek dünya koşullarındaki etkinliğini ve kullanıcı deneyimine etkisini ölçmek amacıyla, geliştirilen Chrome tarayıcı eklentisi ile bir saha testi gerçekleştirilmiştir. Test kapsamında, 100'er URL'den oluşan iki farklı veri seti kullanılmıştır. İlk küme tamamen veri setindeki URL'lerden, ikinci küme ise veri setinden ve dış kaynaklardan alınan karma URL'lerden oluşturulmuştur. Modelin kararlılığını ölçmek için bazı URL'ler iki veri setine dahil edilmiştir.

Saha testinden elde edilen başarı metrikleri ve sistemin çalışma sürelerine ilişkin veriler Tablo 5.3’te sunulmuştur.

Tablo 5.3. Tarayıcı eklentisi saha testi başarı ve süre metrikleri

Metrik	Değer
Toplam URL	177
Doğru Tahmin	176
Yanlış Tahmin	1
Accuracy (%)	99,44
Precision	1
Recall	0,987
F1 Score	0,9935
True Positive	76
True Negative	100
False Positive	0
False Negative	1
Ortalama Süre (ms)	16,517
Minimum Süre (ms)	15,497
Maksimum Süre (ms)	51,476

Tablo 5.3 incelendiğinde, geliştirilen eklentinin %99,44 oranında bir doğruluk (accuracy) ile çalıştığı görülmektedir. Bu sonuç, Bölüm 5.1’de en başarılı model olarak belirlenen XGBoost algoritmasının (%99,76 test başarısı) gerçek zamanlı senaryolarda da tutarlı bir performans sergilediğini göstermektedir. Özellikle Precision (Kesinlik) değerinin 1,000 olması, 100 güvenli sitenin tamamının doğru sınıflandırıldığını ve sistemin hiçbir "False Positive" (yanlış alarm) üretmediğini göstermektedir. Bu durum, eklentinin güvenli siteleri yanlışlıkla engelleyerek kullanıcı deneyimini bozmaması yönündeki tasarım hedefiyle uyum göstermektedir.

Sistemin gerçek zamanlılık kapasitesini belirleyen süre metrikleri incelendiğinde URL yakalama, özellik çıkarımı ve model tahmini süreçlerinin ortalama 16,517 milisaniyede tamamlandığı görülmüştür. En uzun işlem süresi dahi (51,476 ms), insanın algılayabileceğinin çok altında bir süreye karşılık gelmektedir. Bu bulgular, sistemin Bölüm 4.5’te belirtilen "düşük gecikme süresi" ve "gerçek zamanlıya yakın performans" iddialarını deneysel olarak doğrulamaktadır.

Sonuç olarak hafif yapılı lexical analiz yöntemi ve optimize edilmiş XGBoost modelinin birleşimi, sistemin hem yüksek bir tespit başarısına ulaşmasını hem de internette gezinme deneyimini kesintiye uğratmadan arka planda milisaniyeler içerisinde koruma sağlamasını mümkün kılmaktadır.

KAYNAKLAR

- [1] Cybersecurity Ventures. (2023). *Official cybercrime report 2023*. <https://cybersecurityventures.com/cybercrime-to-cost-the-world-8-trillion-annually-in-2023/>
- [2] IBM. (2023). *Cost of a data breach report 2023*. <https://d110erj175o600.cloudfront.net/wp-content/uploads/2023/07/25111651/Cost-of-a-Data-Breach-Report-2023.pdf>
- [3] Verizon. (2023). *Data breach investigations report (DBIR)*. <https://www.verizon.com/business/resources/reports/dbir/>
- [4] Akca, M. F. (2021, 21 Şubat). *Lojistik regresyon nedir, nasıl çalışır?* Medium. <https://mfakca.medium.com/lojistik-regresyon-nedir-nas%C4%B1l-%C3%A7al%C4%B1%C5%9F%C4%B1r-4e1d2951c5c1>
- [5] Lee, F. (t.y.). *Lojistik regresyon nedir?* IBM Think. <https://www.ibm.com/think/topics/logistic-regression>
- [6] Kavlıkoğlu, E. (t.y.). *Destek vektör makinesi (SVM) nedir?* IBM Think. <https://www.ibm.com/think/topics/support-vector-machine>
- [7] Geyik, B. (2021). Detection of phishing websites from URLs by using machine learning methods. *Proceedings of the Sixth International Conference on Inventive Computation Technologies (ICICT)* içinde. IEEE.
- [8] Catak, F. O., Sahinbas, K., & Dörtkardeş, V. (2021). Malicious URL detection using machine learning. A. K. Luhach & A. Elçi (Ed.), *Artificial intelligence paradigms for smart cyber-physical systems* (ss. 160-180. IGI Global.
- [9] Alzboon, M. S., & Abualigah, K. M. (2025). Phishing website detection using machine learning. *Wasit Journal of Computer and Mathematics Science*.
- [10] Tang, L., & Mahmoud, T. (2021). A survey of machine learning-based solutions for phishing website detection. *Computers*, 10(3).
- [11] Zeng, Y. (2018). *Malicious URLs and attachments detection on lexical-based features using machine learning* [Yüksek lisans tezi]. University of Victoria.
- [12] Raja, A. S., Vinodini, R., & Kavitha, A. (2021). Lexical features based malicious URL detection using machine learning techniques. *Materials Today: Proceedings*, 47, 163–166. <https://doi.org/10.1016/j.matpr.2021.04.041>
- [13] Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2021). Machine learning based phishing detection from URLs. *Data*, 3(3), 34. <https://www.mdpi.com/2504-4990/3/3/34>
- [14] Rehman, A. U., Imtiaz, I., Javaid, S., & Muslih, M. (2025). Real-time phishing URL detection using machine learning. *Engineering Proceedings*, 107(108). <https://doi.org/10.3390/engproc2025107108>

- [15] Van Kesteren, A. (2024). *URL living standard*. WHATWG. <https://url.spec.whatwg.org/>
- [16] Stallings, W. (2018). *Network security essentials: Applications and standards* (6. baskı). Pearson Education.
- [17] Alshamrani, A., Myneni, S., Chowdhary, A., & Huang, D. (2019). A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys & Tutorials*, 21(2), 1851–1877.
- [18] Almohaimeed, M., Albalwy, F., Algulaiti, L., & Althubyani, H. (2025). Phishing URL detection using deep learning: A resilient approach to mitigating emerging cybersecurity threats. *Ingénierie des Systèmes d'Information*, 30(5), 1219–1227. <https://doi.org/10.18280/isi.300510>
- [19] Ocak, M. E. (2020, 10 Aralık). *Yapay sinir ağları nedir?* TÜBİTAK Bilim Genç. <https://bilimgenc.tubitak.gov.tr/yapay-sinir-aglari-nedir> adresinden 2 Ağustos 2026 tarihinde alınmıştır.
- [20] Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Qahtani, A., & Lutarokhu, S. (2019). Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7, 41525–41550. <https://doi.org/10.1109/ACCESS.2019.2895334>
- [21] Binary Terms. (t.y.). *Lexical analysis*. <https://binaryterms.com/lexical-analysis.html>
- [22] Yuan, J., Liu, Y., & Yu, L. (2021). A novel approach for malicious URL detection based on the joint model. *Security and Communication Networks*, 2021, 4917016.
- [23] Joshi, A., Lloyd, L., Westin, P., & Seethapathy, S. (2019). Using lexical features for malicious URL detection – A machine learning approach. *arXiv*. <https://arxiv.org/abs/1910.06277>
- [24] Abad, S., Gholamy, H., & Aslani, M. (2023). Classification of malicious URLs using machine learning. *Sensors*, 23, 7760.
- [25] Aljabri, M., Alhaidari, F., Mohammad, R. M. A., Mirza, S., Alhamed, D. H., Altamimi, H. S., & Chrouf, S. M. B. (2022). An assessment of lexical, network, and content-based features for detecting malicious URLs using machine learning and deep learning models. *Computational Intelligence and Neuroscience*, 2022, 3241216.
- [26] He, S., Li, B., Peng, H., Xin, J., & Zhang, E. (2021). An effective cost-sensitive XGBoost method for malicious URLs detection in imbalanced dataset. *IEEE Access*, 9, 93089–93096.
- [27] Maci, A., Santorsola, A., Coscia, A., & Iannaccone, A. (2023). Unbalanced web phishing classification through deep reinforcement learning. *Computers*, 12, 118.
- [28] DR, U. S., Patil, A. & Mohana, M. (2023). Malicious URL detection and classification analysis using machine learning models. *2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)* içinde (ss. 470–476). IEEE.

- [29] Do Xuan, C., Nguyen, H. D., & Tisenko, V. N. (2020). Malicious URL detection based on machine learning. *International Journal of Advanced Computer Science and Applications*, 11.
- [30] Patgiri, R., Katari, H., Kumar, R., & Sharma, D. (2019). Empirical study on malicious URL detection using machine learning. *Distributed computing and internet technology: 15th International Conference, ICDCIT 2019 içinde* (ss. 380–388). Springer.
- [31] Tong, X., Jin, B., Wang, J., Yang, Y., Suo, Q., & Wu, Y. (2023). MM-ConvBERT-LMS: Detecting malicious web pages via multi-modal learning and pre-trained model. *Applied Sciences*, 13, 3327.
- [32] Nagy, N., Aljabri, M., Shaahid, A., Ahmed, A. A., Alnasser, F., Almakramy, L., Alhadab, M., & Alfaddagh, S. (2023). Phishing URLs detection using sequential and parallel ML techniques: Comparative analysis. *Sensors*, 23, 3467.
- [33] Alsaedi, M., Ghaleb, F. A., Saeed, F., Ahmad, J., & Alasli, M. (2022). Cyber threat intelligence-based malicious URL detection model using ensemble learning. *Sensors*, 22, 3373.
- [34] Umer, M., Sadiq, S., Karamti, H., Alhebshi, R. M., Alnowaiser, K., Eshmawi, A., Song, H., & Ashraf, I. (2022). Deep learning-based intrusion detection methods in cyber-physical systems: Challenges and future trends. *Electronics*, 11, 3326.
- [35] Elsadig, M., Ibrahim, A. O., Basheer, S., Alohal, M. A., Alshunaifi, S., Alqahtani, H., Alharbi, N., & Nagmeldin, W. (2022). Intelligent deep machine learning cyber phishing URL detection based on BERT features extraction. *Electronics*, 11, 3647.
- [36] Abdul Samad, S. R., Balasubramanian, S., Al-Kaabi, A. S., Sharma, B., Chowdhury, S., Mehbodniya, A., Webber, J. L., & Bostani, A. (2023). Analysis of the performance impact of fine-tuned machine learning model for phishing URL detection. *Electronics*, 12, 1642.
- [37] Kumi, S., Lim, C., & Lee, S.-G. (2021). Malicious URL detection based on associative classification. *Entropy*, 23, 182.
- [38] Roy, S. S., Awad, A. I., Amare, L. A., Erkihun, M. T., & Anas, M. (2022). Multimodel phishing URL detection using LSTM, bidirectional LSTM, and GRU models. *Future Internet*, 14, 340.
- [39] Ashwini, P., & Vadivelan, N. D. (2021). Security from phishing attack on internet using evolving fuzzy neural network. *CVR Journal of Science and Technology*, 20(1), 50–55.
- [40] Aalla, H. V. S., Dumpala, N. R., & Eliazer, M. (2021). Malicious URL prediction using machine learning techniques. *Annals of the Romanian Society for Cell Biology*, 2170–2176.
- [41] Anil, G. N. (2020). Detection of phishing websites based on feature extraction using machine learning. *International Research Journal of Engineering and Technology (IRJET)*.

- [42] Johnson, C., Khadka, B., Basnet, R. B., & Doleck, T. (2020). Towards detecting and classifying malicious URLs using deep learning. *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications*, 11(4), 31–48.
- [43] Chang, P. (2022). Multi-layer perceptron neural network for improving detection performance of malicious phishing URLs without affecting other attack types classification. *arXiv*. <https://arxiv.org/abs/2203.00774>
- [44] Tariq, H. A., Yang, W., Hameed, I., Ahmed, B., & Khan, R. U. (2017). Using black-list and white-list technique to detect malicious URLs. *International Journal of Innovative Research in Information Security (IJIRIS)*, 4, 1–7.
- [45] Wang, Y. (2022). Malicious URL detection: An evaluation of feature extraction and machine learning algorithm. *Highlights in Science, Engineering and Technology*, 23, 117–123.
- [46] Wei, B., Hamad, R. A., Yang, L., He, X., Wang, H., Gao, B., & Woo, W. L. (2019). A Deep-Learning-Driven Light-Weight Phishing Detection Sensor. *Sensors*, 19(19), 4258. <https://doi.org/10.3390/s19194258>
- [47] Hajaj, C., Hason, N., & Dvir, A. (2022). Less is more: Robust and novel features for malicious domain detection. *Electronics*, 11(6), 969.
- [48] Fotiadou, K., Velivassaki, T. H., Voulkidis, A., Skias, D., Tsekeridou, S., & Zahariadis, T. (2021). Network traffic anomaly detection via deep learning. *Information*, 12(5), 215.
- [49] Almuhaideb, A. M., Aslam, N., Alabdullatif, A., Altamimi, S., Alothman, S., Alhussain, A., Aldosari, W., Alsunaidi, S. J., & Alissa, K. A. (2022). Homoglyph attack detection model using machine learning and hash function. *Journal of Sensor and Actuator Networks*, 11(3), 54. <https://doi.org/10.3390/jsan11030054>
- [50] Saxe, J., & Berlin, K. (2017). eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys. *arXiv*. <https://arxiv.org/abs/1702.08568>
- [51] Lee, S., & Kim, J. (2012). WarningBird: Detecting suspicious URLs in Twitter stream. *Network and Distributed System Security Symposium (NDSS)* içinde.
- [52] Tung, S. P., Wong, K. Y., Kuzminykh, I., Bakhshi, T., & Ghita, B. (2021). Using a machine learning model for malicious URL type detection. *International Conference on Next Generation Wired/Wireless Networking* içinde (ss. 493–505). Springer.
- [53] Jain, A. K., & Gupta, B. B. (2016). A novel approach to protect against phishing attacks at client side using auto-updated white-list. *EURASIP Journal on Information Security*, 2016, 1–11.
- [54] Alsariera, Y. A., Adeyemo, V. E., Balogun, A. O., & Alazzawi, A. K. (2020). AI meta-learners and extra-trees algorithm for the detection of phishing websites. *IEEE Access*, 8, 142532–142542.
- [55] Liu, L., Ren, W., Xie, F., Yi, S., Yi, J., & Jia, P. (2021). Learning-based detection for malicious android application using code vectorization. *Security and Communication Networks*, 2021, 1–11.

- [56] Diwan, T. D., Choubey, S., Hota, H.S., Goyal, S.B., Jamal, S.S., Shukla, P.K., & Tiwari, B. (2021). Feature entropy estimation (FEE) for malicious IoT traffic and detection using machine learning. *Mobile Information Systems*, 2021, 1–13.
- [57] Wang, Y., Cai, W., Lyu, P., & Shao, W. (2018). A combined static and dynamic analysis approach to detect malicious browser extensions. *Security and Communication Networks*, 2018.
- [58] Khan, N., Abdullah, J., & Khan, A. S. (2017). Defending malicious script attacks using machine learning classifiers. *Wireless Communications and Mobile Computing*, 2017.
- [59] Zhao, H., Chang, Z., Bao, G., & Zeng, X. (2019). Malicious domain names detection algorithm based on N-gram. *Journal of Computer Networks and Communications*, 2019, 1–9.
- [60] Ispahany, J., & Islam, R. (2020). Detecting malicious URLs of COVID-19 pandemic using ML technologies. *arXiv*. <https://arxiv.org/abs/2009.09224>
- [61] Yu, X. (2020). Phishing websites detection based on hybrid model of deep belief network and support vector machine. *IOP Conference Series: Earth and Environmental Science*, 602(1), 012001.
- [62] Rafsanjani, A. S., Kamaruddin, N. B., Behjati, M., Aslam, S., Sarfaraz, A., & Amphawan, A. (2024). Enhancing malicious URL detection: A novel framework leveraging priority coefficient and feature evaluation. *IEEE Access*.
- [63] Nazeeruddin, E., Latif, G., & Mohammad, N. (2024). Malicious URL detection and categorization using machine learning techniques. *2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)* içinde. IEEE.
- [64] Jaswanthi, T., Harshitha, T., Belwal, M., & Tanya, S. (2024). Malicious URL detection: Comparative study of machine learning algorithms. *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)* içinde. IEEE.
- [65] Dinakar, D. S., & Deepak, K. (2024). Shallow and deep learning based approaches for malicious URL detection: A comprehensive performance evaluation. *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)* içinde. IEEE.
- [66] Alharbi, A., Alsubhi, K., & Alotaibi, R. (2025). Lightweight malicious URL detection using deep learning. *Scientific Reports*, 15. <https://doi.org/10.1038/s41598-025-26653-2>
- [67] Türk, F., & Kılıçaslan, M. (2025). Malicious URL detection with advanced machine learning and optimization-supported deep learning models. *Applied Sciences*, 15, 10090.
- [68] Onwudebelu, U., Ugah, J. O., Ezech, S. E., & Fasola, O. O. (2026). Securing online platforms: Hybrid machine learning approaches for URL phishing detections. *Journal of Intelligent Communication*, 5(1), 24–46.
- [69] Yi, L., Omotosho, A., & Balogun, H. (2026). Phishing URL detection and interpretability with machine learning: A cross-dataset approach. *Security and Privacy*, 9, e70175. <https://doi.org/10.1002/spy2.70175>

- [70] Jha, T., Goswami, H., Solanki, C., Nagal, D., Yadav, V., & Prasad, A. (2025). *StealthPhisher* [Veri seti]. Mendeley Data. <https://doi.org/10.17632/m2479kmybx.1>

ÖZGEÇMİŞ

Ad-Soyad : Mustafa Melih TÜFEKCİOĞLU

ÖĞRENİM DURUMU:

- **Lisans** : 2023, Sakarya Üniversitesi, Bilgisayar ve Bilişim Bilimleri Fakültesi, Bilgisayar Mühendisliği Bölümü
- **Yüksek lisans** : Devam ediyor, Sakarya Üniversitesi, Bilgisayar ve Bilişim Mühendisliği Anabilim Dalı, Siber Güvenlik Programı

MESLEKİ DENEYİM:

- 2024 yılında özel bir havacılık firmasında yazılım geliştirme mühendisi olarak çalışmaya başladı.